线性模型与非线性扩展

- 引言
- 线性回归
- 线性回归实例
- 线性模型的非线性拓展
- 通用的多类分类器

引言

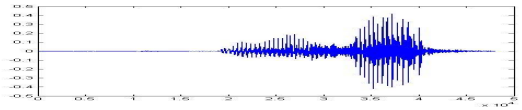
- 机器学习的基本流程



- 回归与分类是两大类问题
  - 回归问题的输出是连续值
  - 分类问题的输出是离散值
  - 区别：模型最后一层的设计方式不同

机器学习 ≈ 寻找函数

- Speech Recognition

$f($  $)=$ “How are you”

- Image Recognition

$f($  $)=$ “Cat”

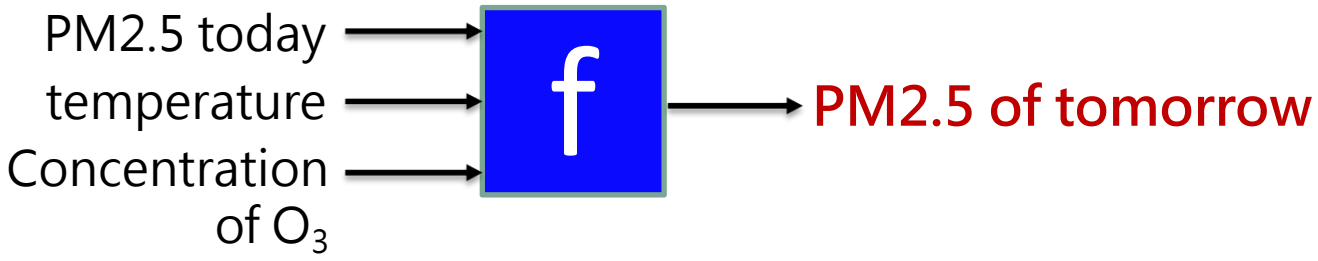
- Playing Go

$f($  $)=$ “5-5” (next move)

函数类型

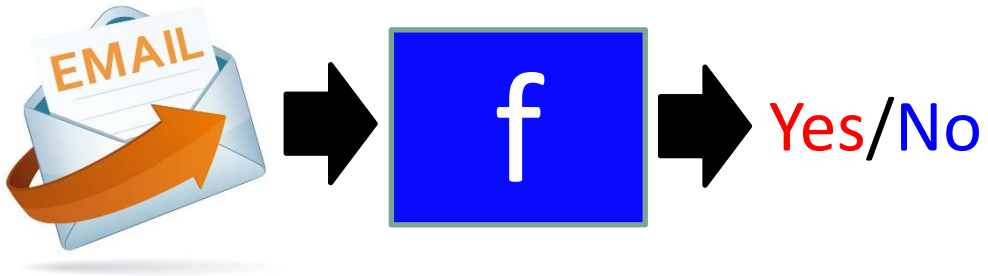
Regression: 函数输出一个标量值，连续值

Predict PM2.5



Classification: 函数输出类别的种类，离散值

Spam filtering

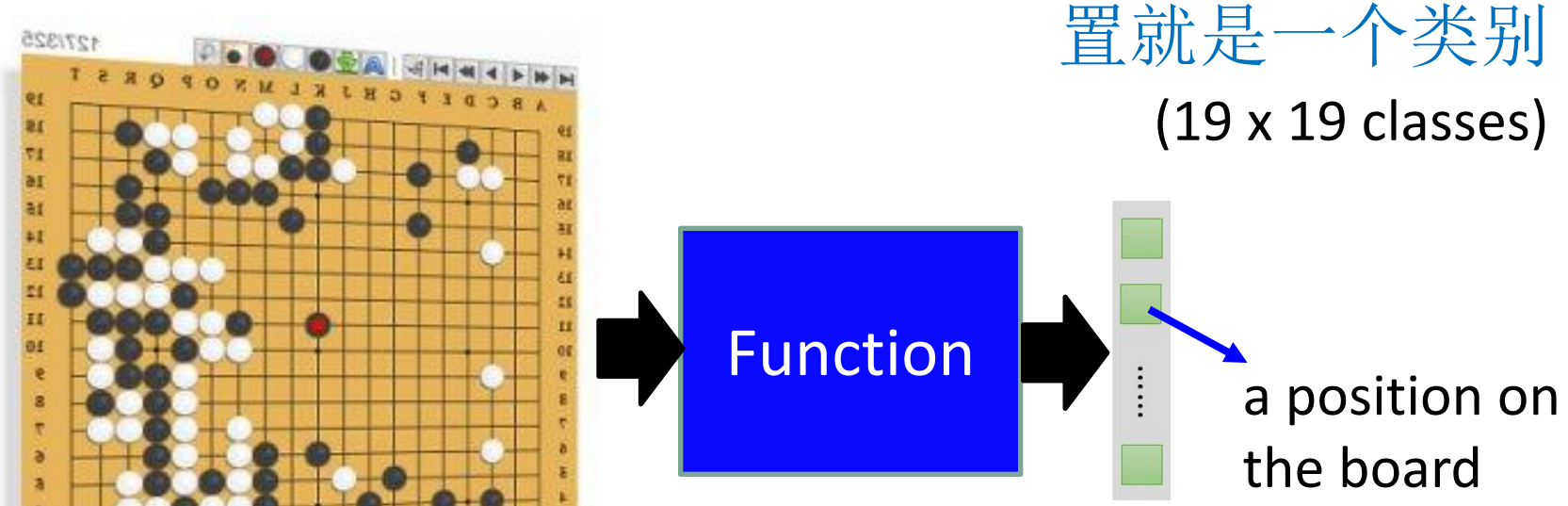


函数类型

Classification: 函数输出类别的种类，离散值

每一次落子的位置就是一个类别

(19 x 19 classes)



Playing GO

线性回归

- 给定由m个属性描述的样本$\boldsymbol{x}=(x_1;x_2;\ldots;x_m)$，其中$x_i$是$\boldsymbol{x}$在第$i$个属性上的取值，线性回归（linear regression）试图学得一个通过属性值的线性组合来进行预测的函数：

$$f(\boldsymbol{x})=w_1x_1+w_2x_2+\ldots+w_mx_m+b$$

- 一般用向量的形式写成：

$$f(\boldsymbol{x})=\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}+b$$

其中$\boldsymbol{w}=(w_1;w_2;\ldots;w_m)$。

- 给定训练数$D = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \ldots, (\boldsymbol{x}_i, y_i), \ldots, (\boldsymbol{x}_n, y_n)\}$
  其中$\boldsymbol{x}_i = (x_{i1}; x_{i2}; \ldots; x_{im})$, $y_i \in \mathbb{R}$

- 可用最小二乘法 (least square method) 对$\mathbf{w}$ 和b进行估计。

- 下面以一元线性回归为例，来详细讲解 w 和 b 的最小二乘法估计

$$f(x_i) = wx_i + b, \text{ 使得 } f(x_i) \simeq y_i$$

- 最小二乘法就是基于预测值和真实值的均方差最小化的方法来估计参数 w 和 b：

$$\begin{aligned}(w^*, b^*) &= \arg\min_{(w,b)} \sum_{i=1}^{n} \left(f\left(x_i\right) - y_i\right)^2 \\ &= \arg\min_{(w,b)} \sum_{i=1}^{n} \left(y_i - wx_i - b\right)^2\end{aligned}$$

- 最小化均方误差

$$E_{(w,b)} = \sum_{i=1}^{n} (y_i - wx_i - b)^2$$

- 分别对 w 和 b 求偏导，可得

$$\frac{\partial E_{(w,b)}}{\partial w} = 2\left(w\sum_{i=1}^{n} x_i^2 - \sum_{i=1}^{n}(y_i - b)\,x_i\right)$$

$$\frac{\partial E_{(w,b)}}{\partial b} = 2\left(nb - \sum_{i=1}^{n}(y_i - wx_i)\right)$$

- 令上两式为零可得到 w 和 b 最优解的闭式(closed-form）解：

$$w = \frac{\sum_{i=1}^{n} y_i (x_i - \bar{x})}{\sum_{i=1}^{n} x_i^2 - \frac{1}{n}\left(\sum_{i=1}^{n} x_i\right)^2}$$

$$b = \frac{1}{n}\sum_{i=1}^{n} (y_i - w x_i)$$

其中，$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$

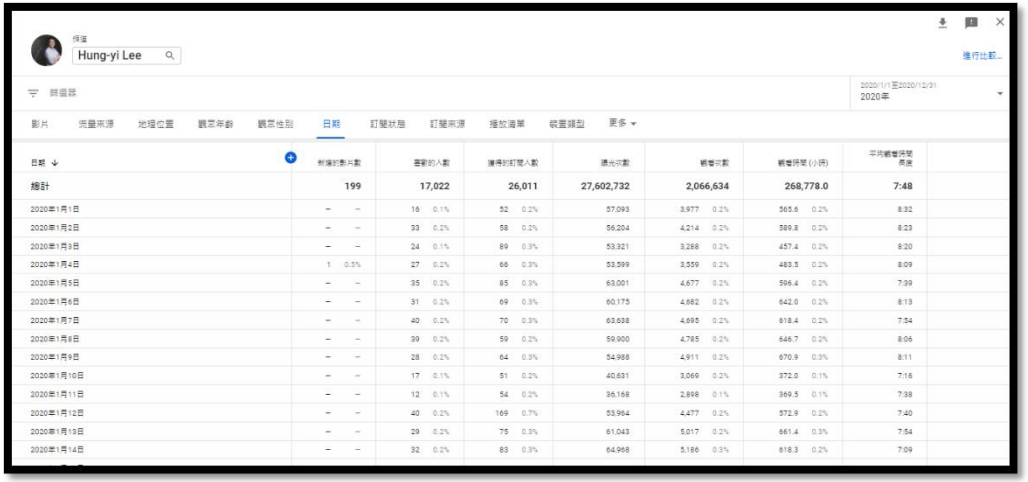
实际应用中一般不采用显式求解的方法，往往通过优化方法来进行迭代寻找最优参数。比如梯度下降法

线性回归实例

- 目标：视频网站的特定视频观看次数预测
- 特征：连续若干天的观看次数分布
- 预测：明天的观看次数

- 机器学习步骤
  1. 定义一个带有未知参数的函数（模型）
  2. 定义损失函数
  3. 基于优化方法得到最优参数

1. 定义一个带有未知参数的函数（模型）

$y = f($  $)$

Model $y = b + wx_1$ 模型基于领域知识构建

feature

y: no. of views on 2/26, x_1: no. of views on 2/25

w and b are unknown parameters (learned from data)

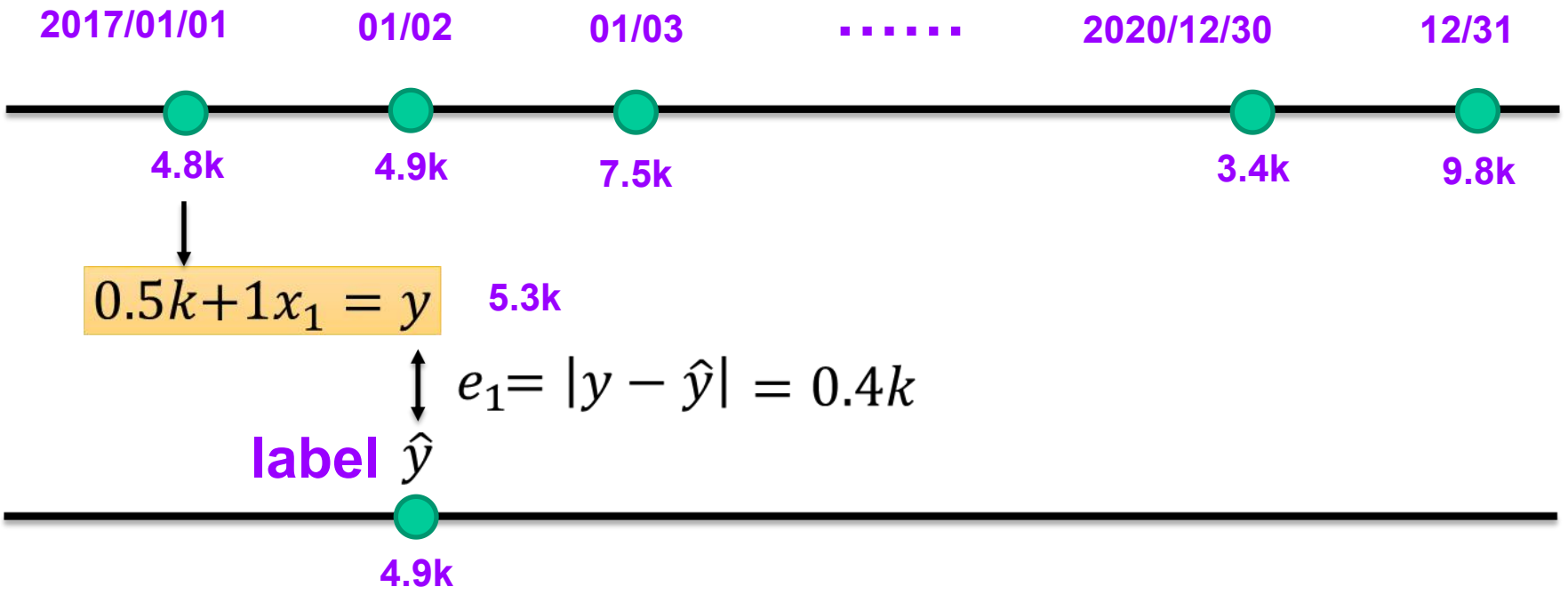
weight **bias**

2. 定义 Loss
基于Training Data

- Loss is a function of parameters $L(b, w)$
- Loss: how good a set of values is.

$L(0.5k, 1)$ $y = b + wx_1 \longrightarrow y = 0.5k + 1x_1$ How good it is?

Data from 2017/01/01 – 2020/12/31

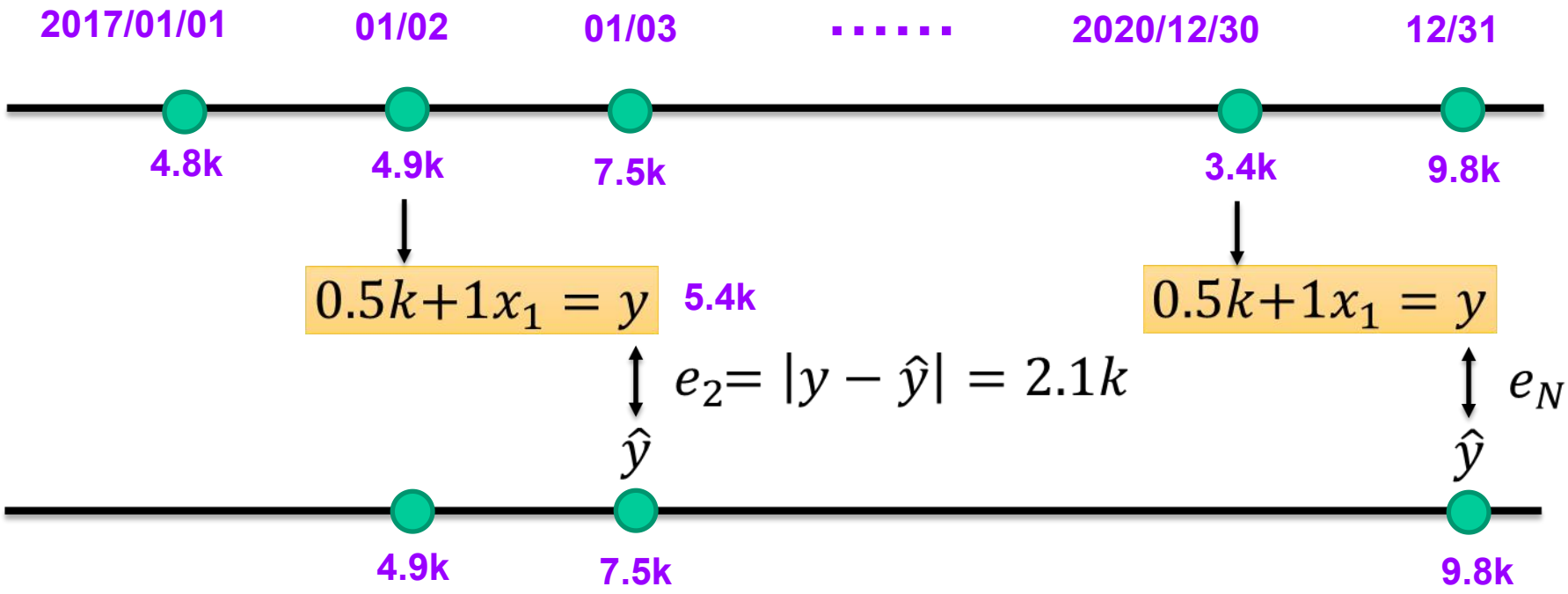


2. 定义 Loss
基于Training Data

- Loss is a function of parameters $L(b, w)$
- Loss: how good a set of values is.

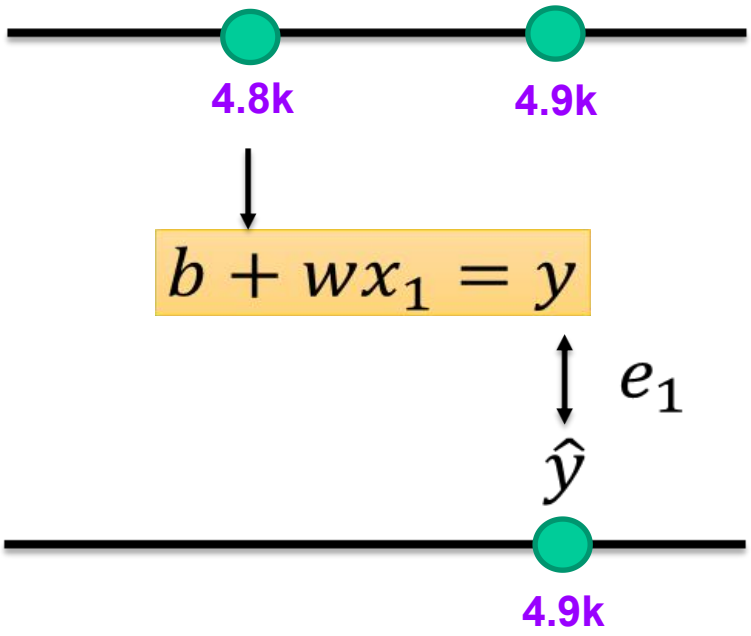
$L(0.5k, 1)$ $y = b + wx_1 \longrightarrow y = 0.5k + 1x_1$ How good it is?

Data from 2017/01/01 – 2020/12/31



2. 定义 Loss
基于Training Data

- Loss is a function of parameters $L(b, w)$
- Loss: how good a set of values is.



Loss: $L = \frac{1}{N}\sum_n e_n$

$e = |y - \hat{y}|$ L is mean absolute error (MAE)

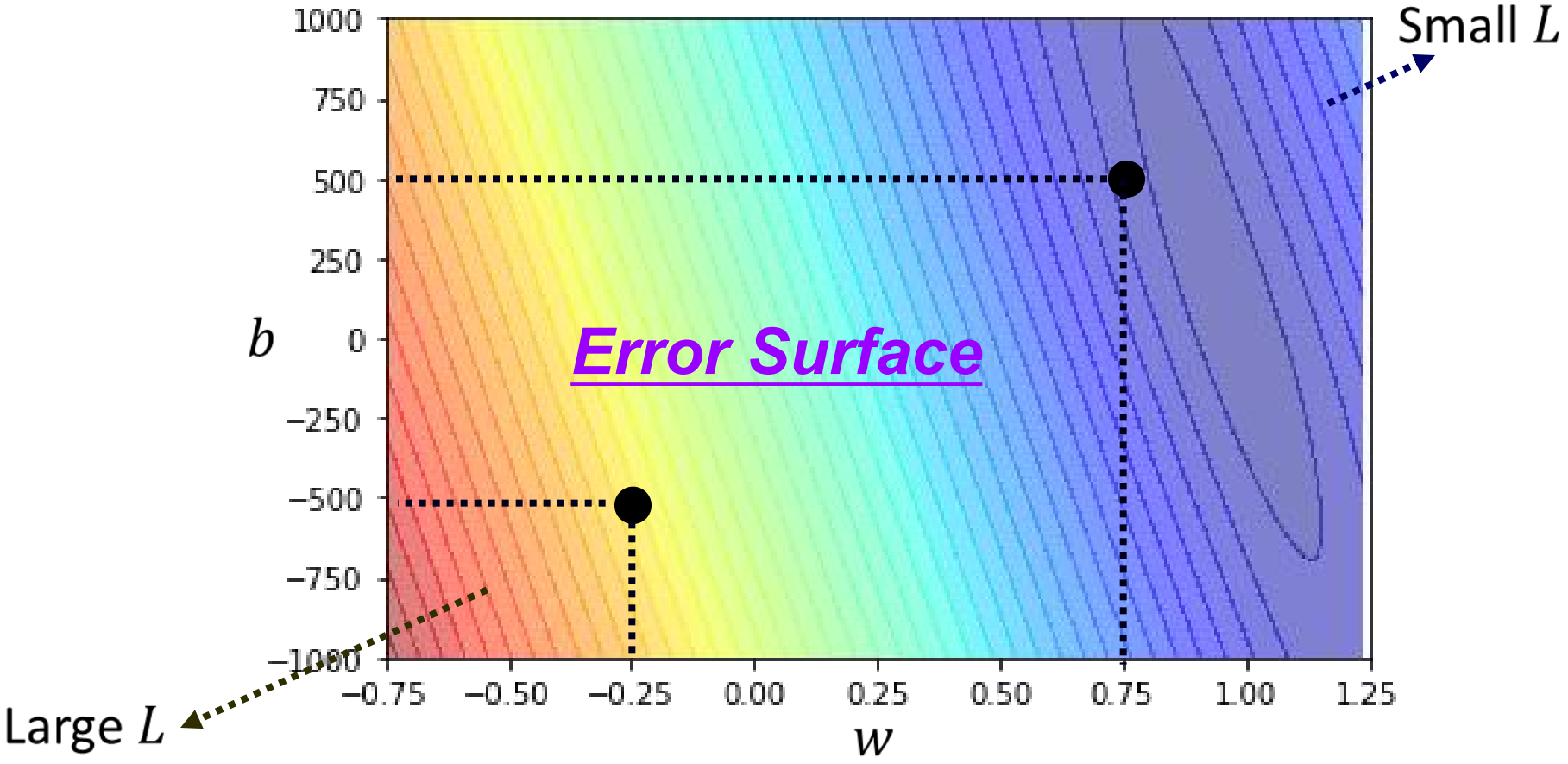
$e = (y - \hat{y})^2$ L is mean square error (MSE)

If y and $\hat{y}$ are both probability distributions → **Cross-entropy**

2. 定义 Loss
基于Training Data

- **Loss is a function of parameters** $L(b, w)$
- **Loss: how good a set of values is.**

Model $y = b + wx_1$

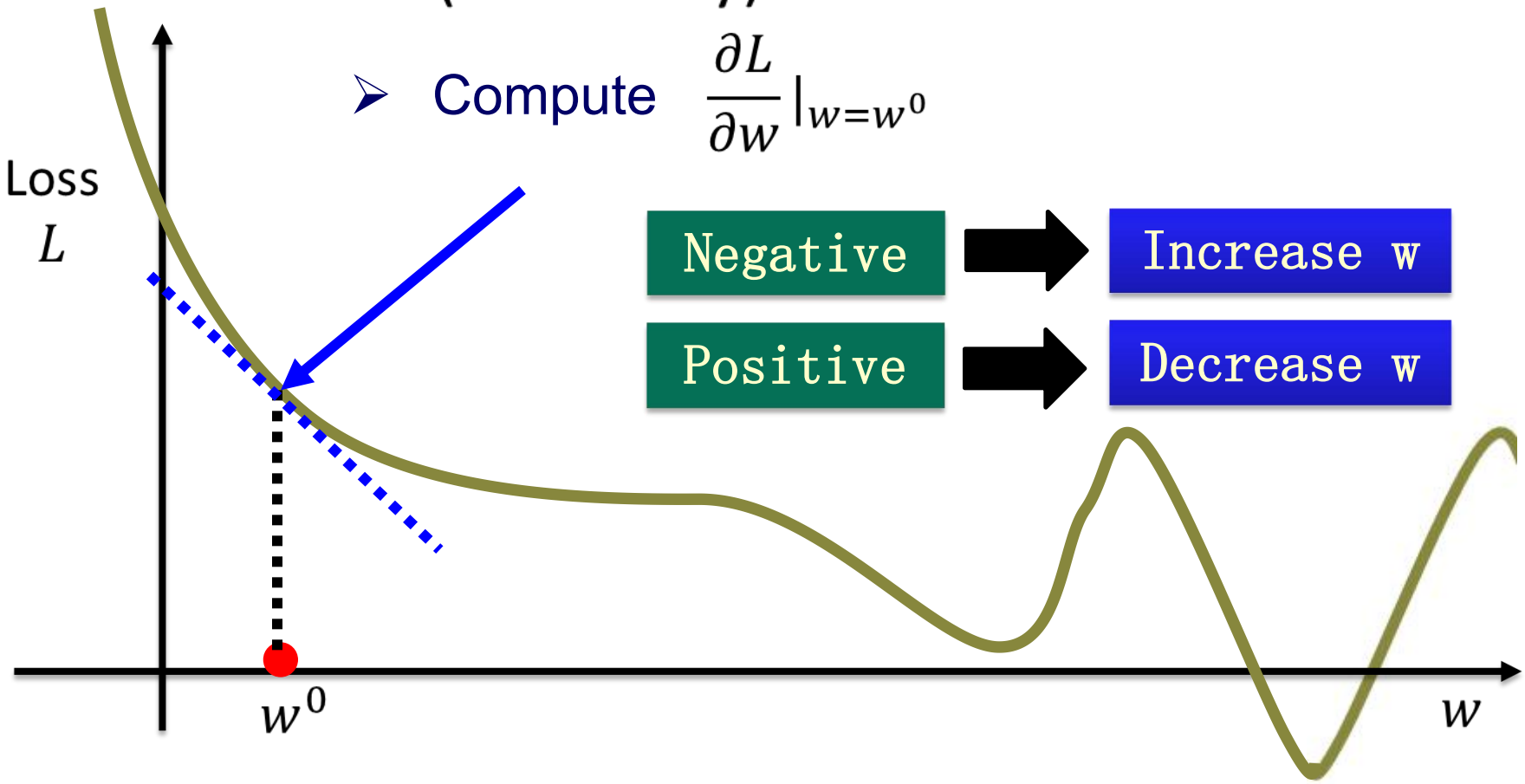


3. 优化

$$w^* = arg \min_w L$$

Gradient Descent

- (Randomly) Pick an initial value w^0
- Compute $\frac{\partial L}{\partial w}|_{w=w^0}$



3. 优化

$$w^* = arg \min_w L$$

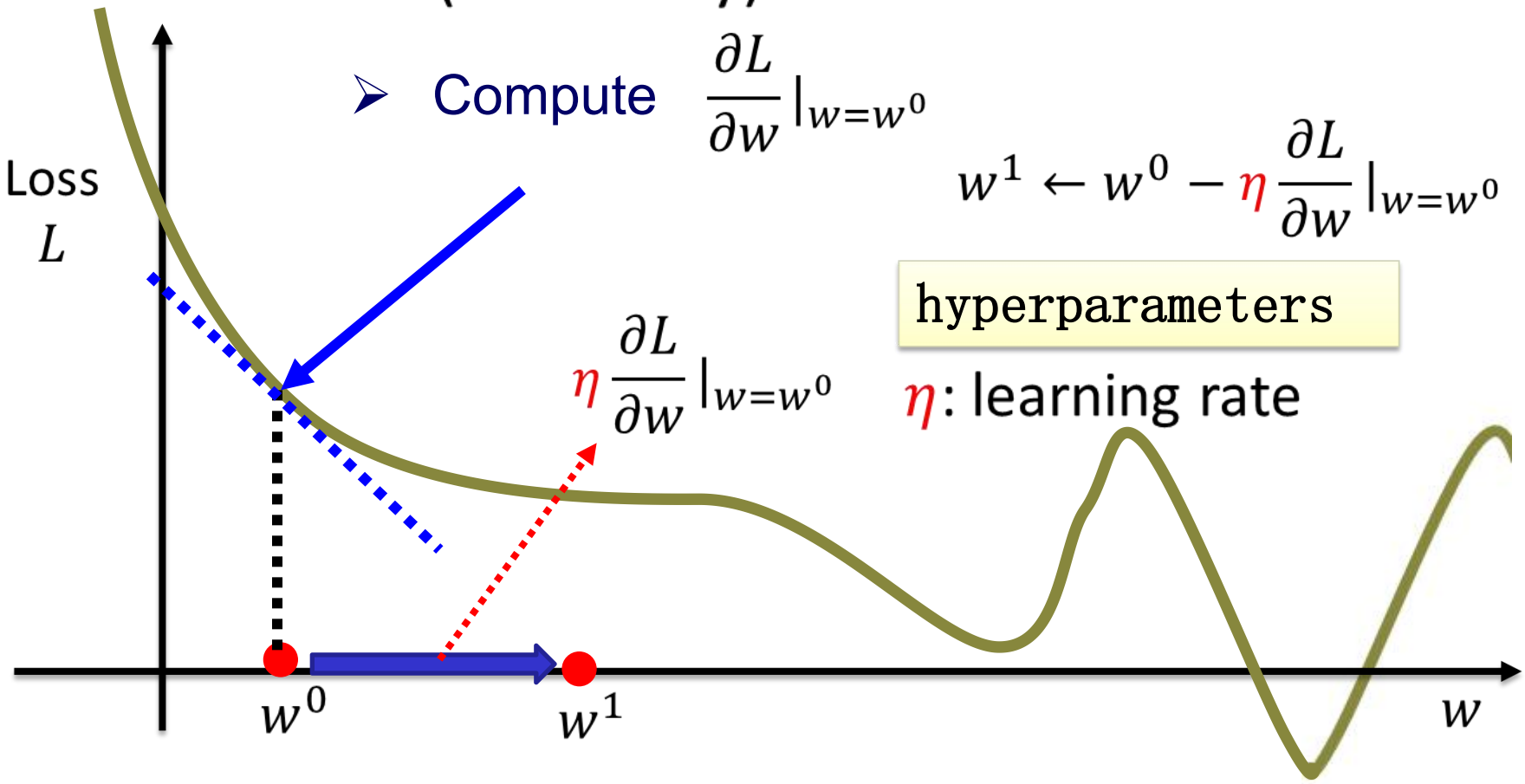
Gradient Descent

- (Randomly) Pick an initial value w^0
- Compute $\frac{\partial L}{\partial w}|_{w=w^0}$

$$w^1 \leftarrow w^0 - \eta \frac{\partial L}{\partial w}|_{w=w^0}$$

hyperparameters

η: learning rate



3. 优化

$$w^* = arg \min_w L$$

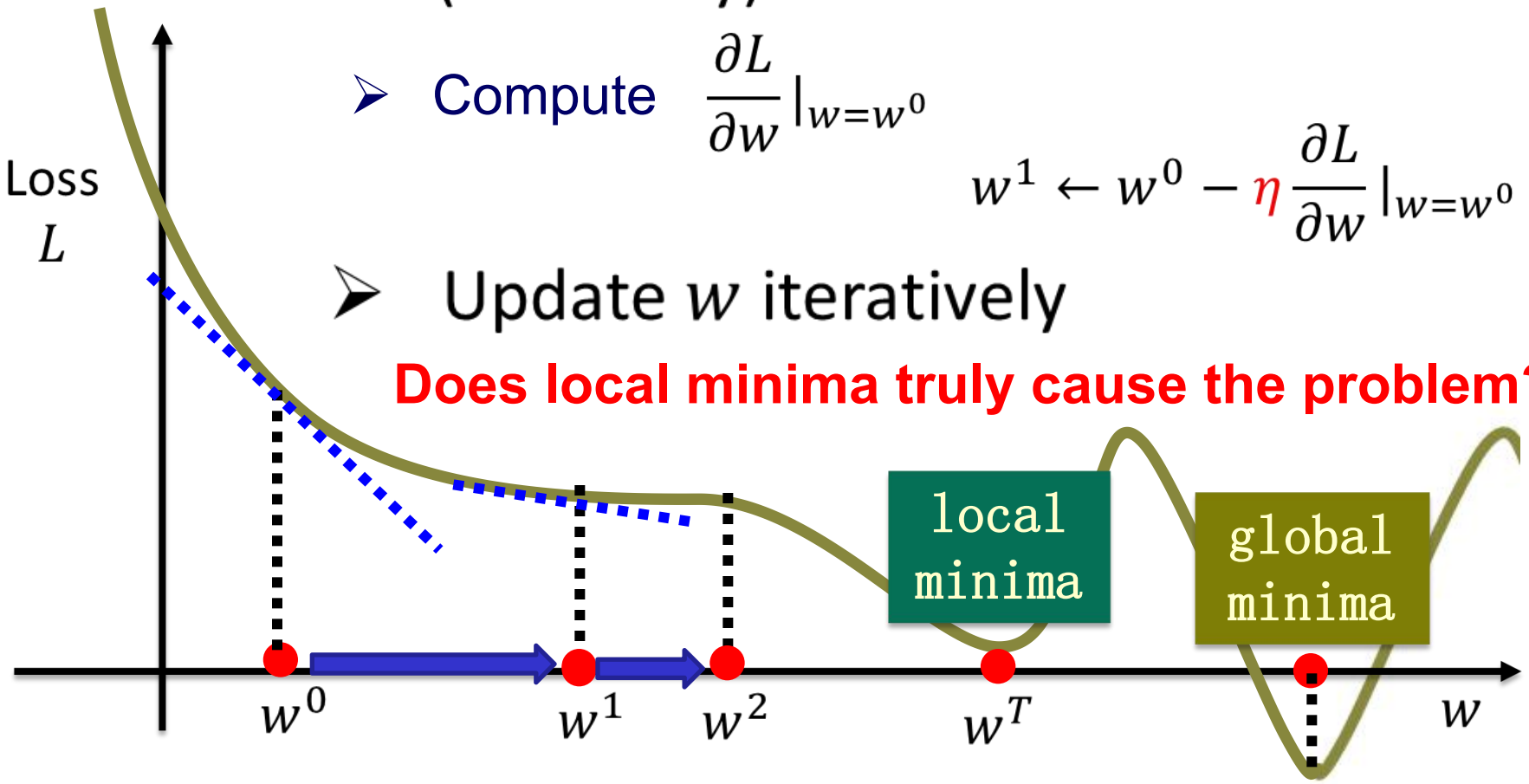
Gradient Descent

- (Randomly) Pick an initial value w^0
- Compute $\frac{\partial L}{\partial w}|_{w=w^0}$

$$w^1 \leftarrow w^0 - \eta \frac{\partial L}{\partial w}|_{w=w^0}$$

- Update w iteratively

Does local minima truly cause the problem?



3. 优化

$$w^*, b^* = arg \min_{w,b} L$$

- (Randomly) Pick initial values w^0, b^0
- Compute

$$\frac{\partial L}{\partial w}\Big|_{w=w^0,b=b^0} \qquad w^1 \leftarrow w^0 - \eta \frac{\partial L}{\partial w}\Big|_{w=w^0,b=b^0}$$

$$\frac{\partial L}{\partial b}\Big|_{w=w^0,b=b^0} \qquad b^1 \leftarrow b^0 - \eta \frac{\partial L}{\partial b}\Big|_{w=w^0,b=b^0}$$

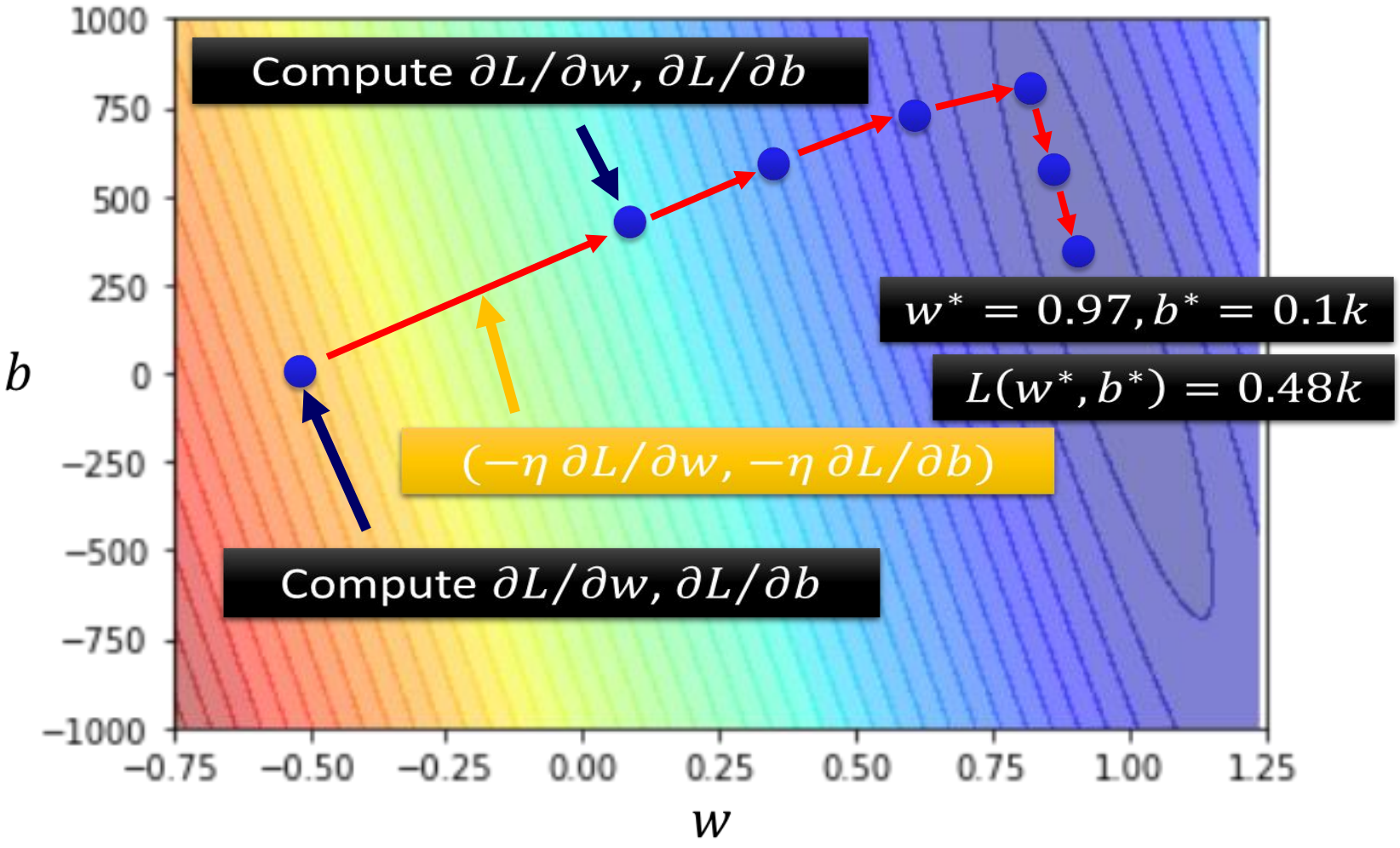
Can be done in one line in most deep learning frameworks

- Update w and b interatively

3. 优化

Model $y = b + wx_1$

$$w^*, b^* = arg \min_{w,b} L$$



Machine Learning is so simple ……

$y = b + wx_1$

$w^* = 0.97, b^* = 0.1k$

$L(w^*, b^*) = 0.48k$

Step 1: 定义函数（带有未知参数）

Step 2: 定义损失(Loss)（基于训练数据）

Step 3: 优化（Optimization）

Training

$y = 0.1k + 0.97x_1$ achieves the smallest loss $L = 0.48k$ on data of 2017 – 2020 (**training data**)

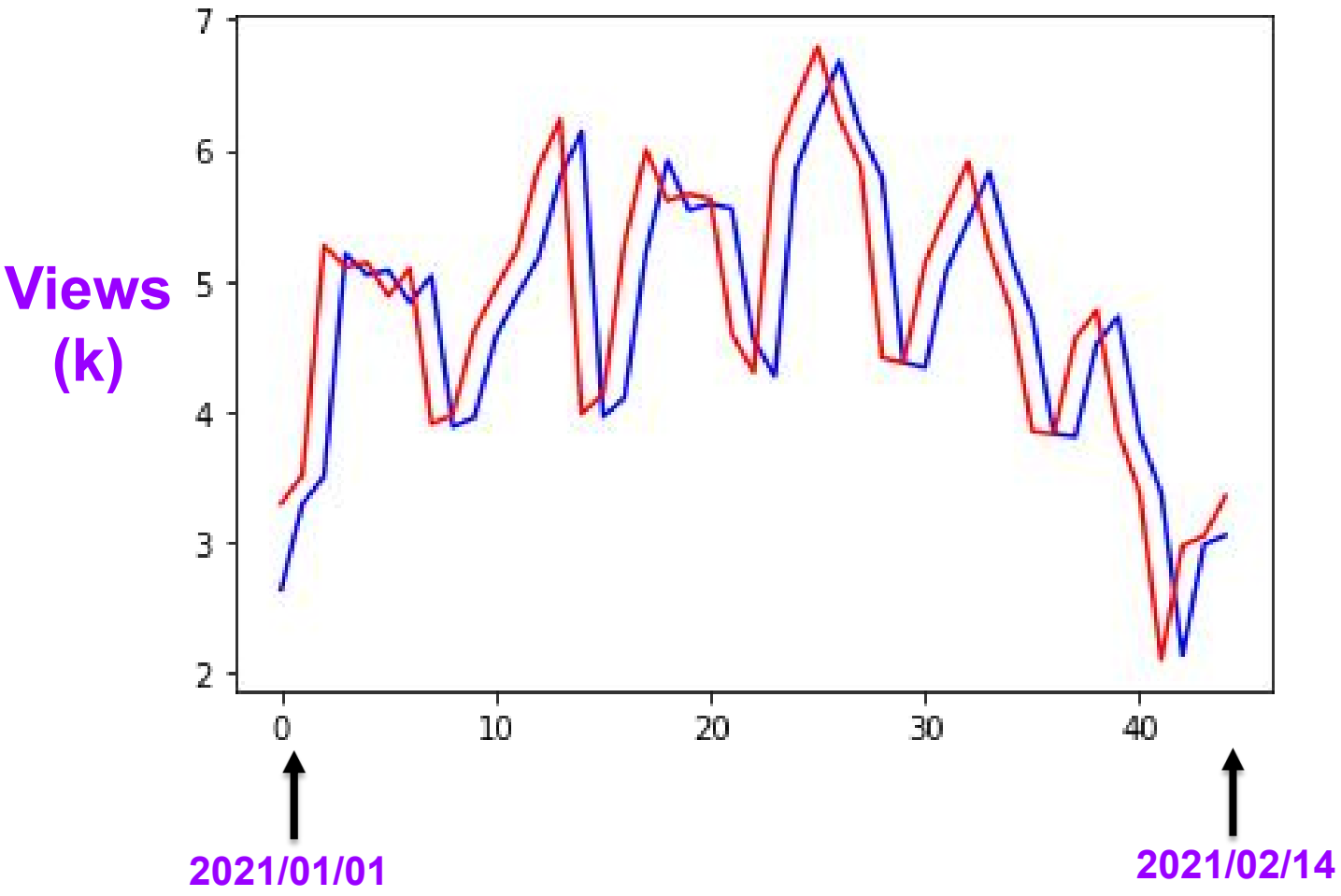
How about data of 2021 (unseen during training)?

$L' = 0.58k$

$$y = 0.1k + 0.97x_1$$

Red: 标签值

blue: 估计值

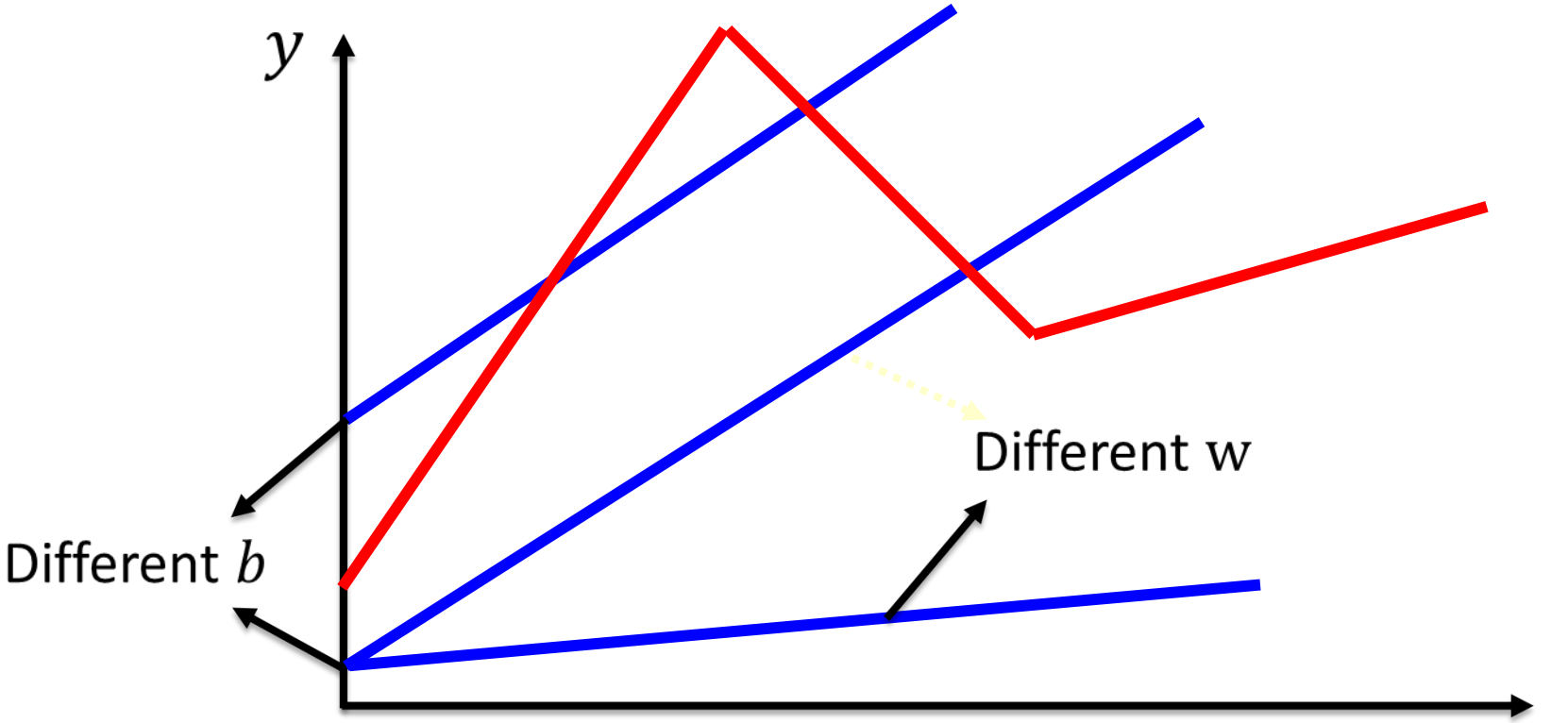


	2017 - 2020	2021
$y = b + wx_1$	$L = 0.48k$	$L' = 0.58k$
$y = b + \sum_{j=1}^{7} w_j x_j$	$L = 0.38k$	$L' = 0.49k$
$y = b + \sum_{j=1}^{28} w_j x_j$	$L = 0.33k$	$L' = 0.46k$
$y = b + \sum_{j=1}^{56} w_j x_j$	$L = 0.32k$	$L' = 0.46k$

Linear models

线性模型的非线性拓展

线性模型过于简单，我们需要更复杂的模型

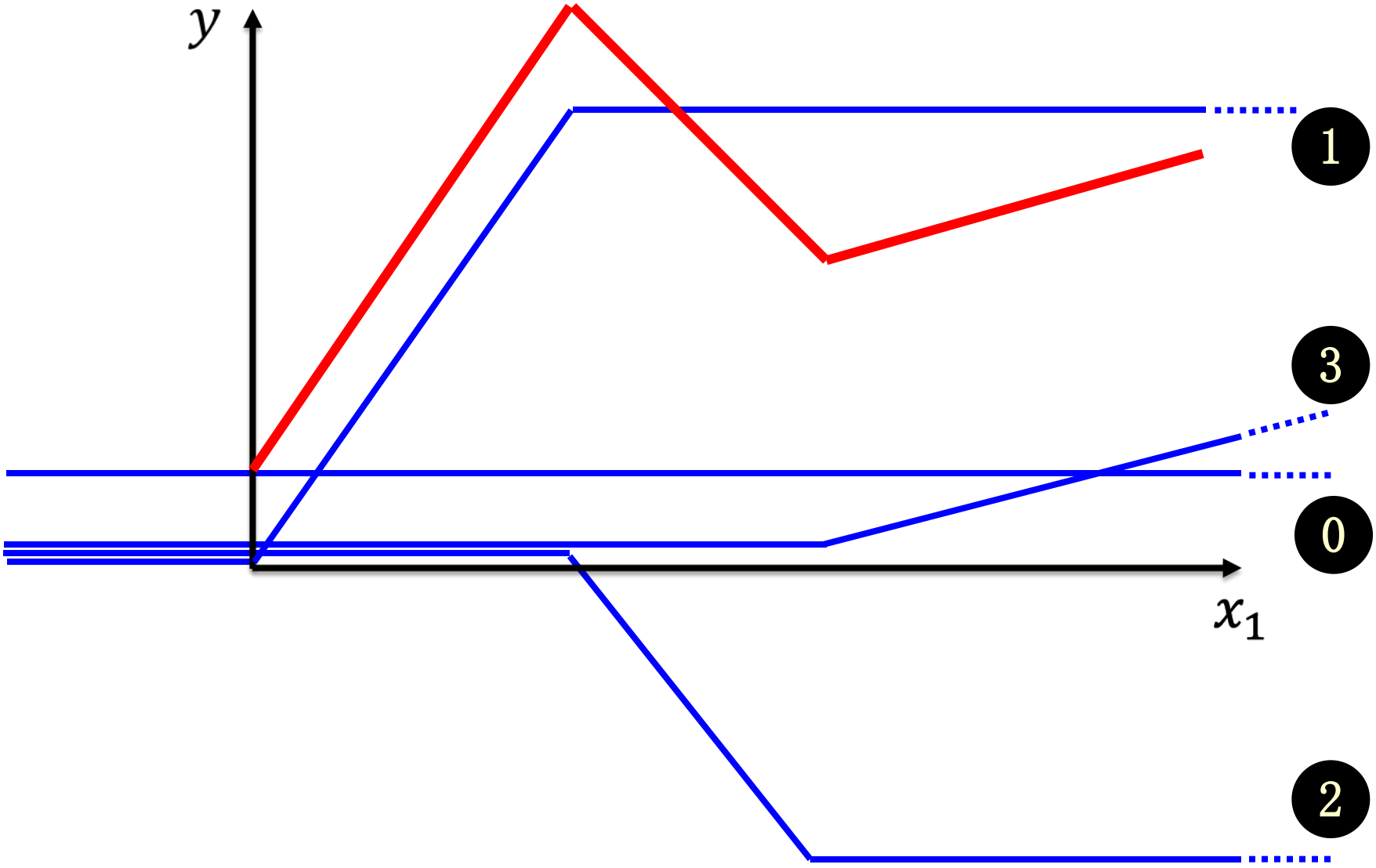


线性模型有非常严重的局限性！无法有效拟合红色的折线。

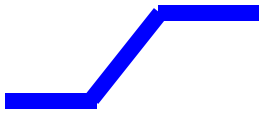
Model Bias

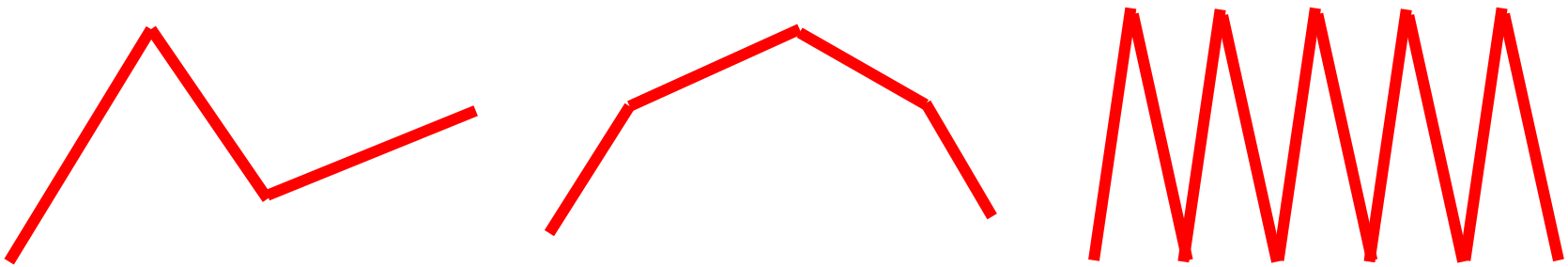
我们需要更加柔性的模型!

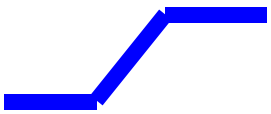
red curve = constant + sum of a set of



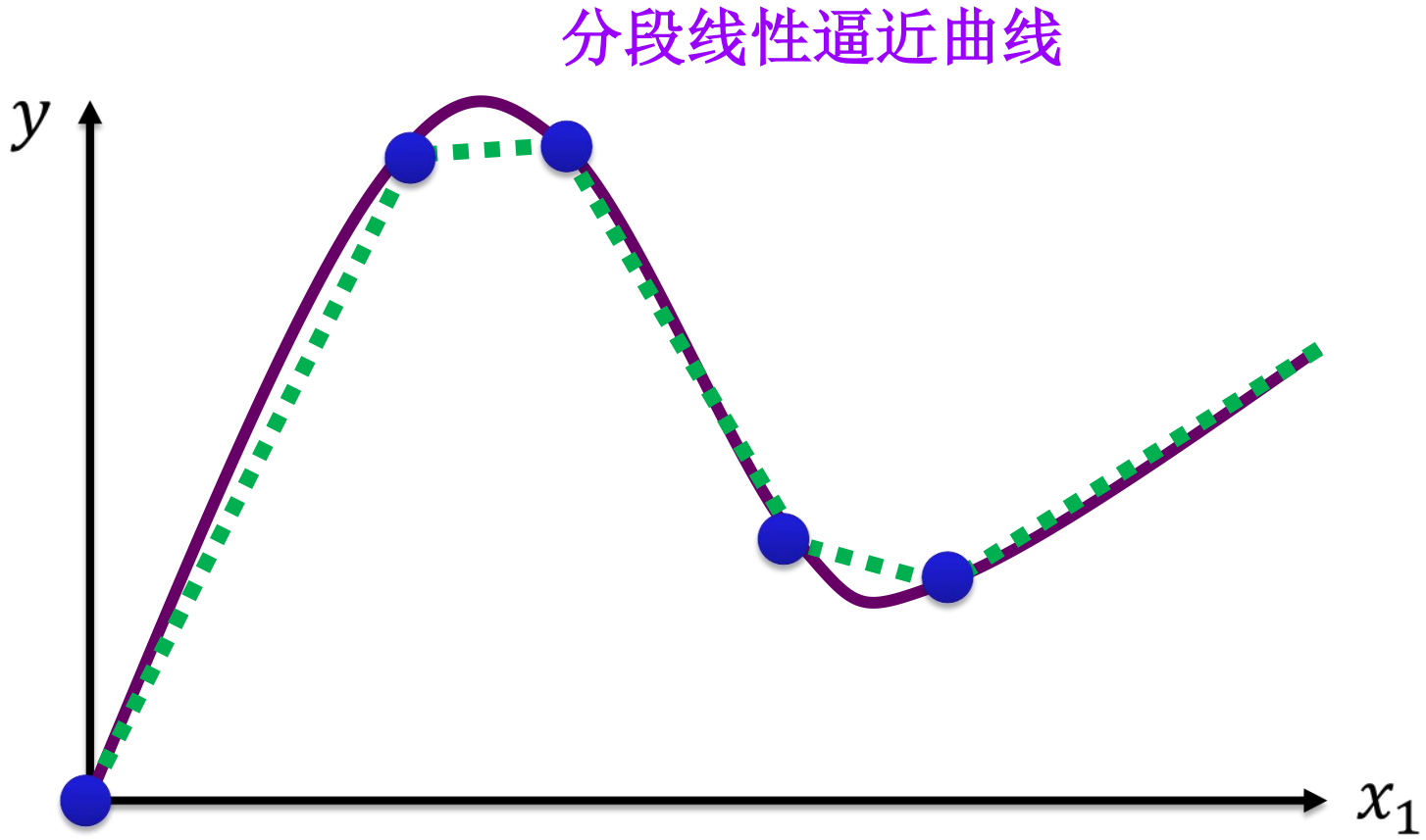
分段线性曲线

= constant + sum of a set of 

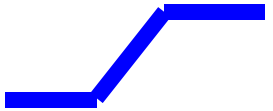


需要更多的 

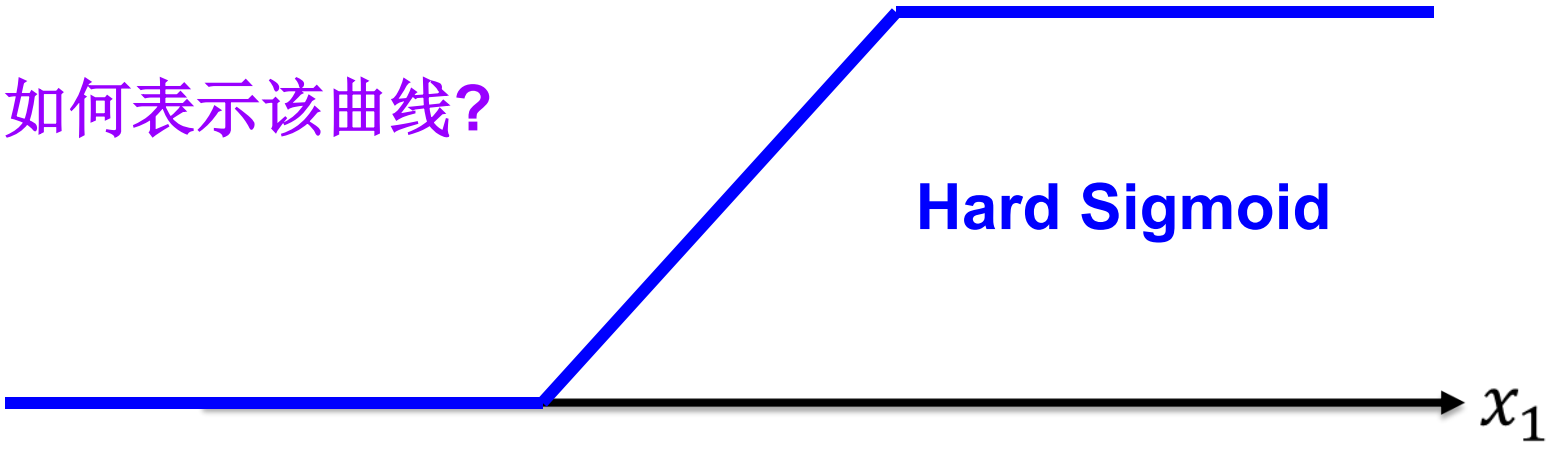
超越分段线性复杂度？ 曲线！



为了有更好的拟合效果，需要更多的片段！

red curve = constant + sum of a set of 

如何表示该曲线?



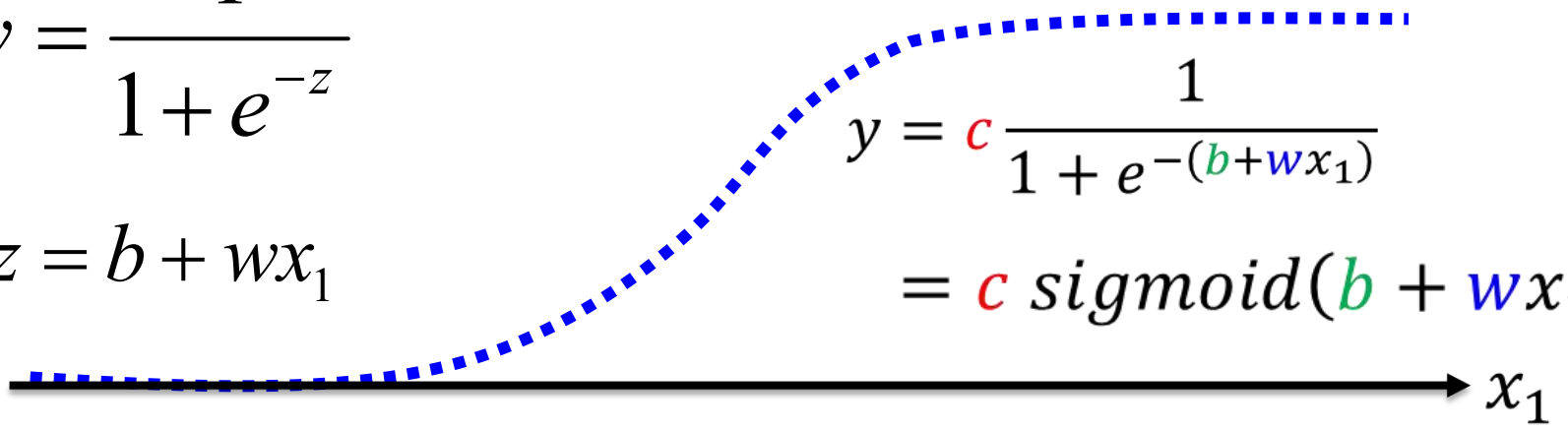
Sigmoid Function

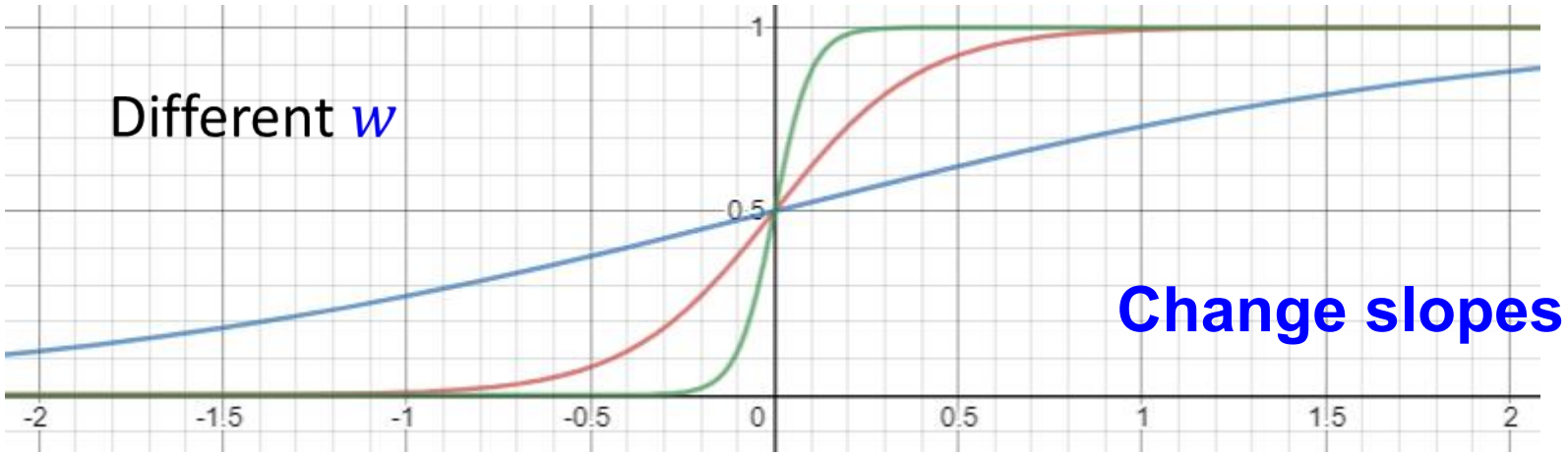
$$y = \frac{1}{1+e^{-z}}$$

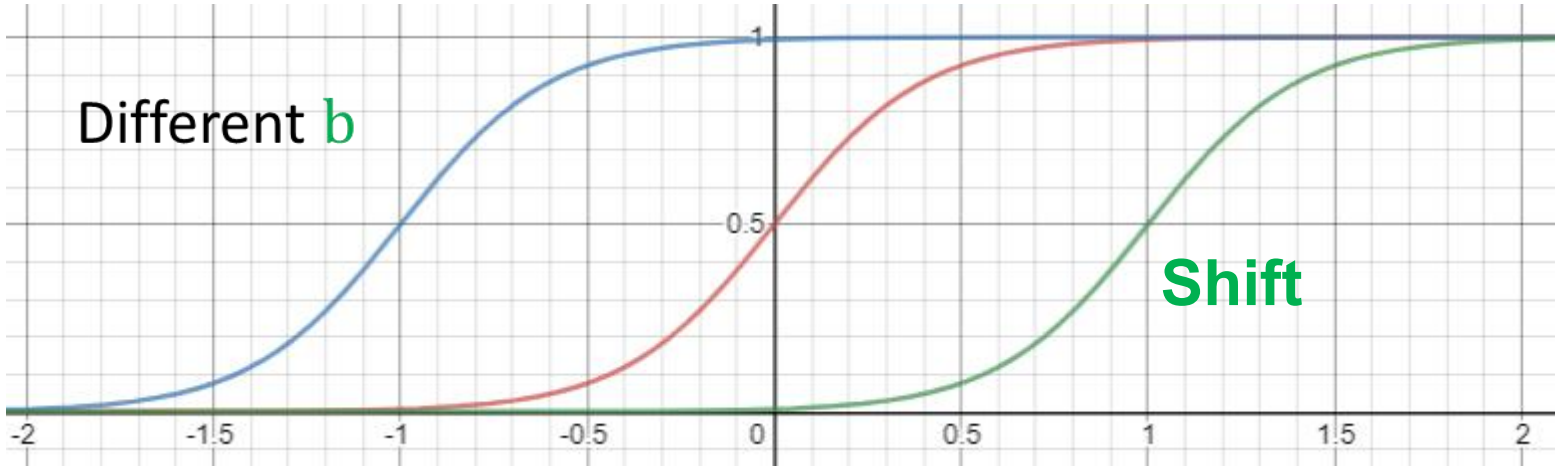
$$z = b + wx_1$$

$$y = c\frac{1}{1+e^{-(b+wx_1)}}$$

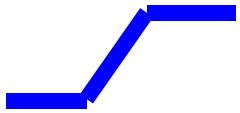
$$= c\ sigmoid(b + wx_1)$$

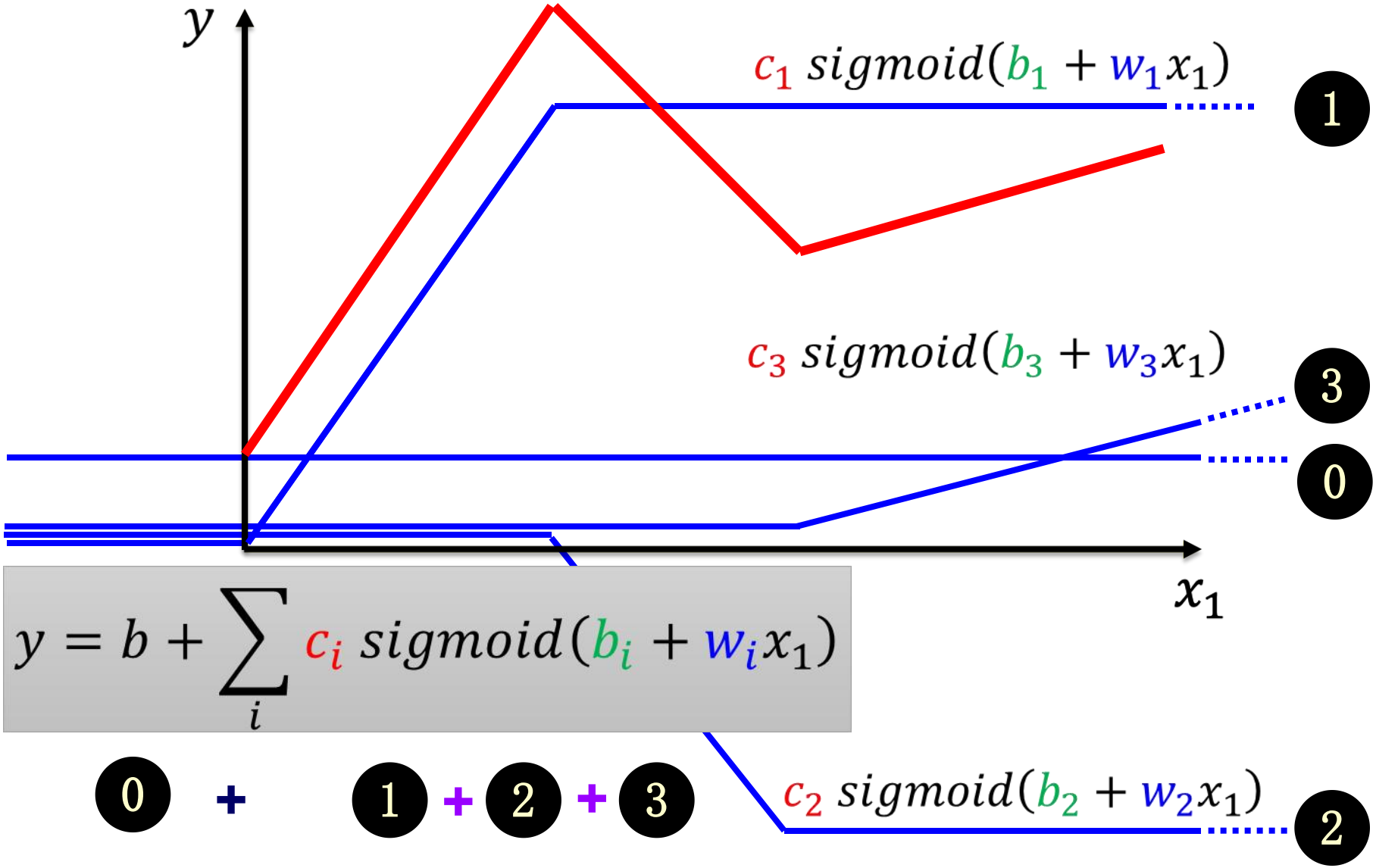



Different w
Change slopes
1
0.5
-2
-1.5
-1
-0.5
0
0.5
1
1.5
2


Different b
Shift
1
0.5
-2
-1.5
-1
-0.5
0
0.5
1
1.5
2


Different c
Change height
1
0.5
-2
-1.5
-1
-0.5
0
0.5
1
1.5
2

red curve = sum of a set of  + constant



新模型：更多特征

$$y = b + wx_1$$

$$y = b + \sum_i c_i \, sigmoid(b_i + w_i x_1)$$

$$y = b + \sum_j w_j x_j$$

$$y = b + \sum_i c_i \, sigmoid\left(b_i + \sum_j w_{ij} x_j\right)$$

$$y = b + \sum_i c_i \, sigmoid\left(b_i + \sum_j w_{ij} x_j\right)$$

j: 1,2,3 **no. of features**

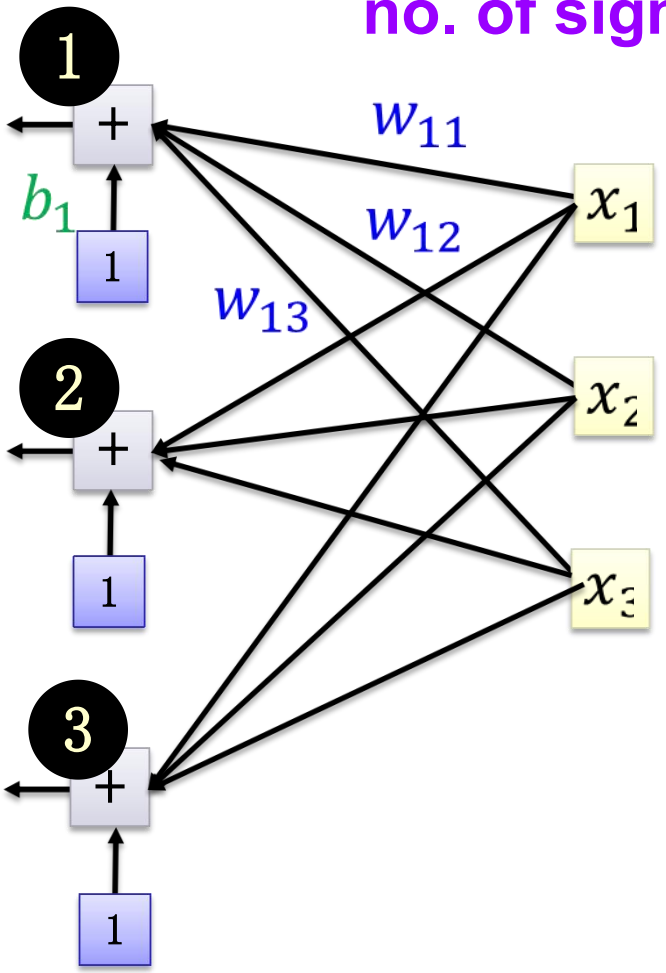
i: 1,2,3 **no. of sigmoid**

$$r_1 = b_1 + w_{11}x_1 + w_{12}x_2 + w_{13}x_3$$

w_{ij}: weight for x_j for i-th sigmoid

$$r_2 = b_2 + w_{21}x_1 + w_{22}x_2 + w_{23}x_3$$

$$r_3 = b_3 + w_{31}x_1 + w_{32}x_2 + w_{33}x_3$$



$$y = b + \sum_i c_i \, sigmoid\left(b_i + \sum_j w_{ij} x_j\right) \qquad i: 1,2,3 \quad j: 1,2,3$$

$$r_1 = b_1 + w_{11}x_1 + w_{12}x_2 + w_{13}x_3$$

$$r_2 = b_2 + w_{21}x_1 + w_{22}x_2 + w_{23}x_3$$

$$r_3 = b_3 + w_{31}x_1 + w_{32}x_2 + w_{33}x_3$$

$$\begin{bmatrix} r_1 \\ r_2 \\ r_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} + \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

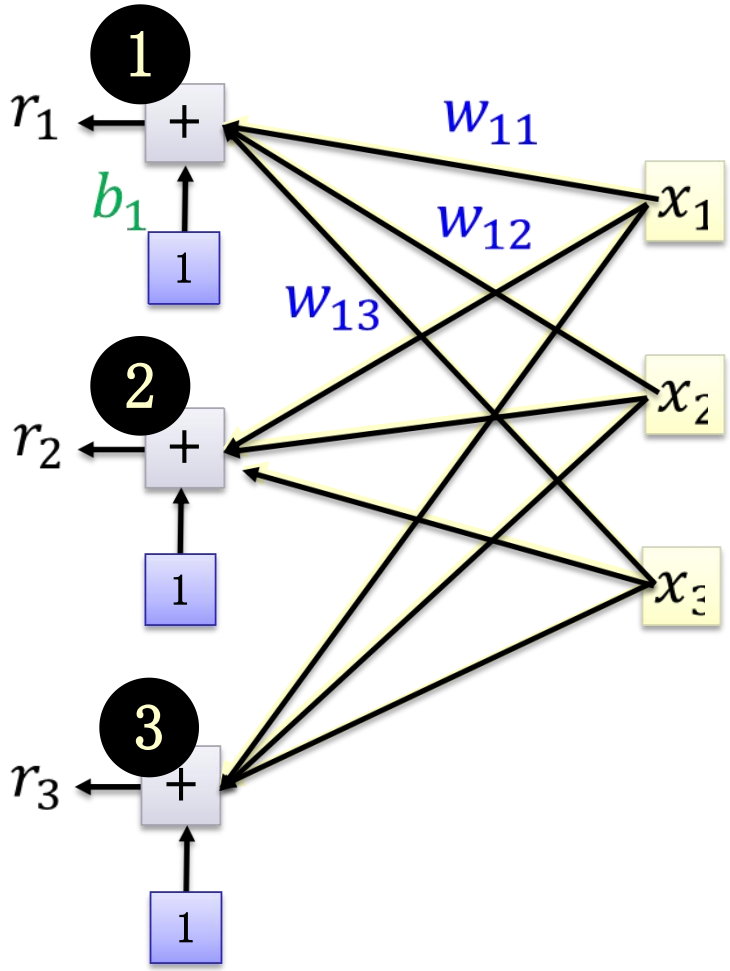
$$\boldsymbol{r} = \boldsymbol{b} + W \boldsymbol{x}$$

$$y = b + \sum_i c_i \, sigmoid\left(b_i + \sum_j w_{ij} x_j\right)$$

i: 1,2,3

j: 1,2,3

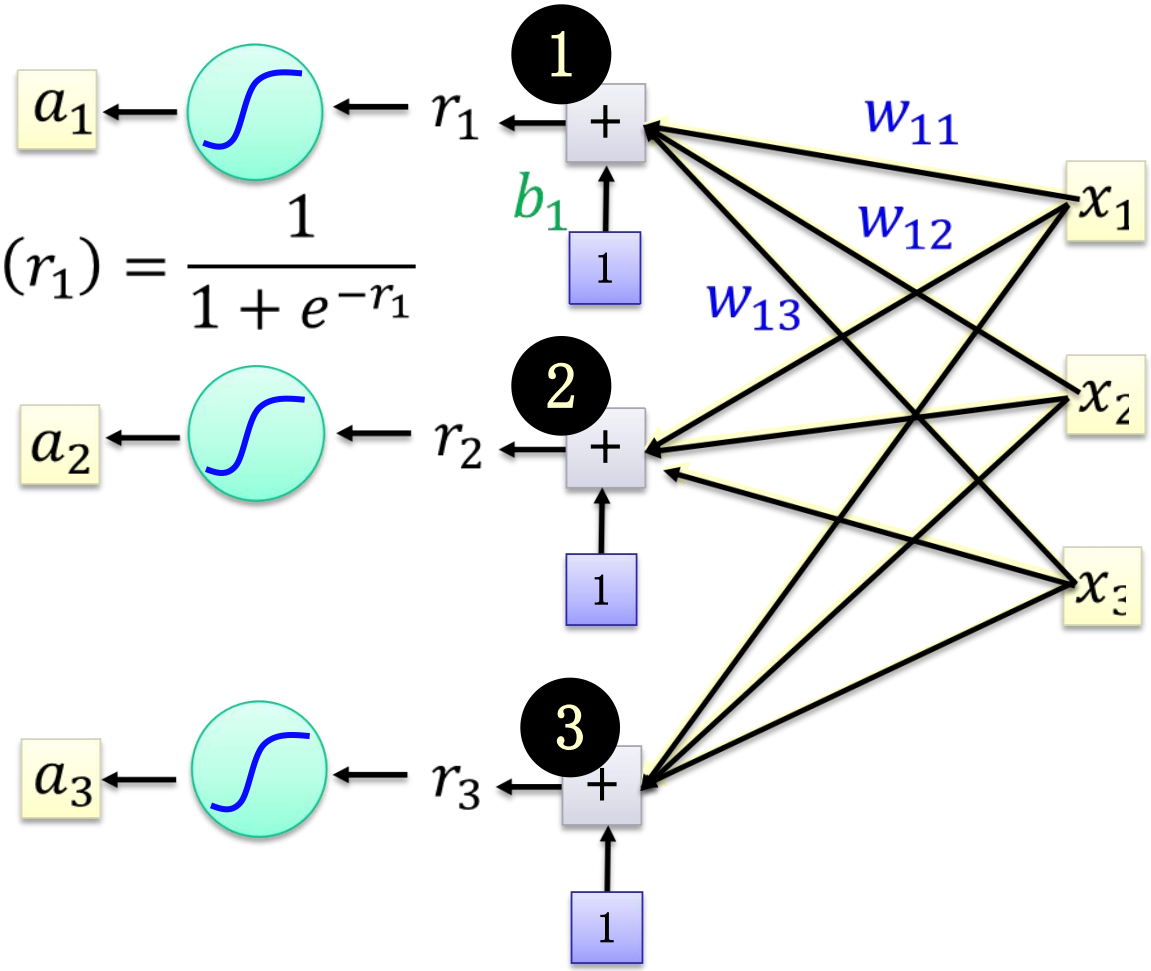
$$\boldsymbol{r} = \boldsymbol{b} + W \boldsymbol{x}$$



$$y = b + \sum_i c_i \, sigmoid\left(b_i + \sum_j w_{ij} x_j\right)$$

i: 1,2,3

j: 1,2,3



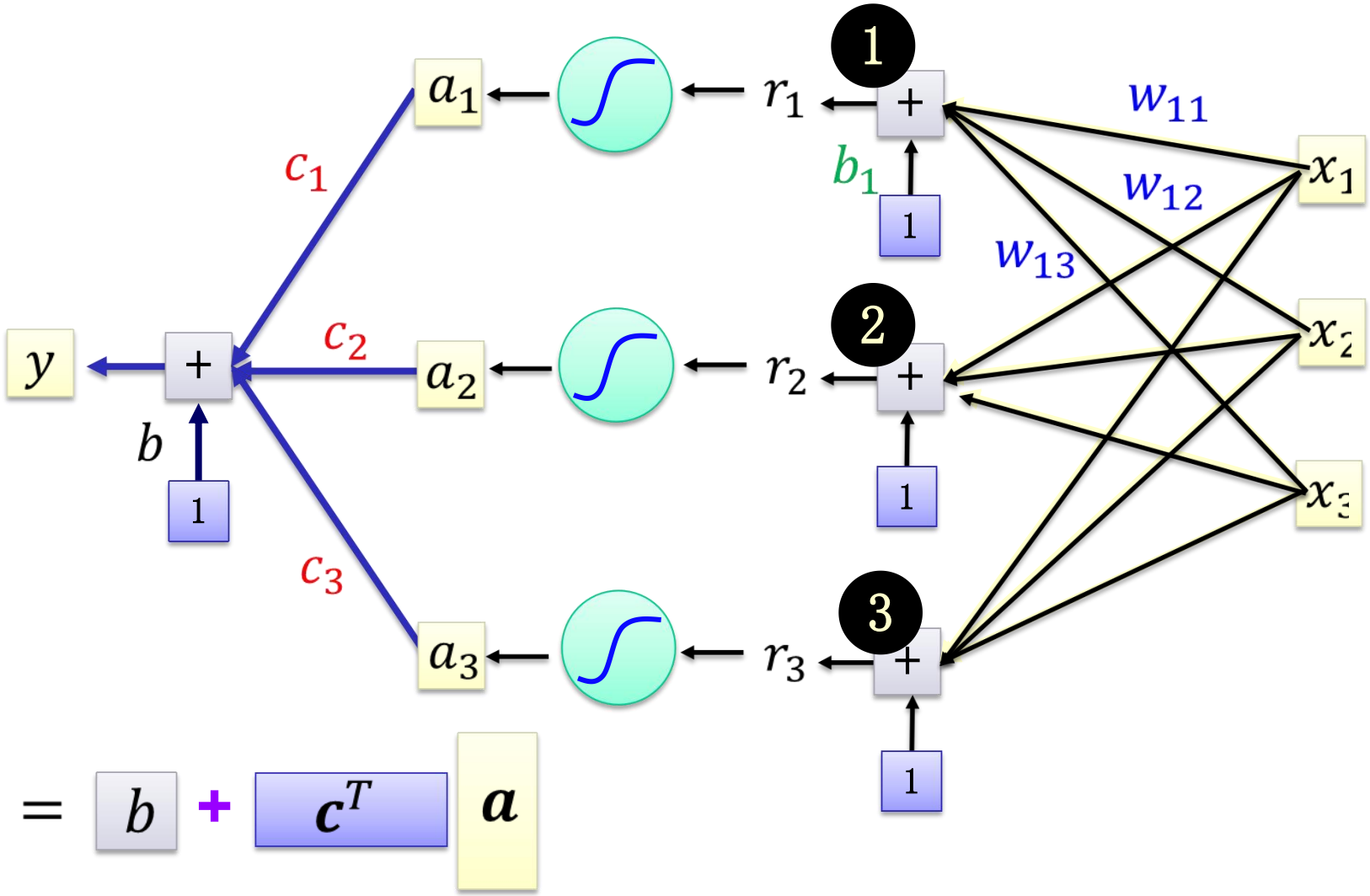
$$a_1 = sigmoid(r_1) = \frac{1}{1 + e^{-r_1}}$$

$$\boldsymbol{a} = \sigma(\ \boldsymbol{r}\ )$$

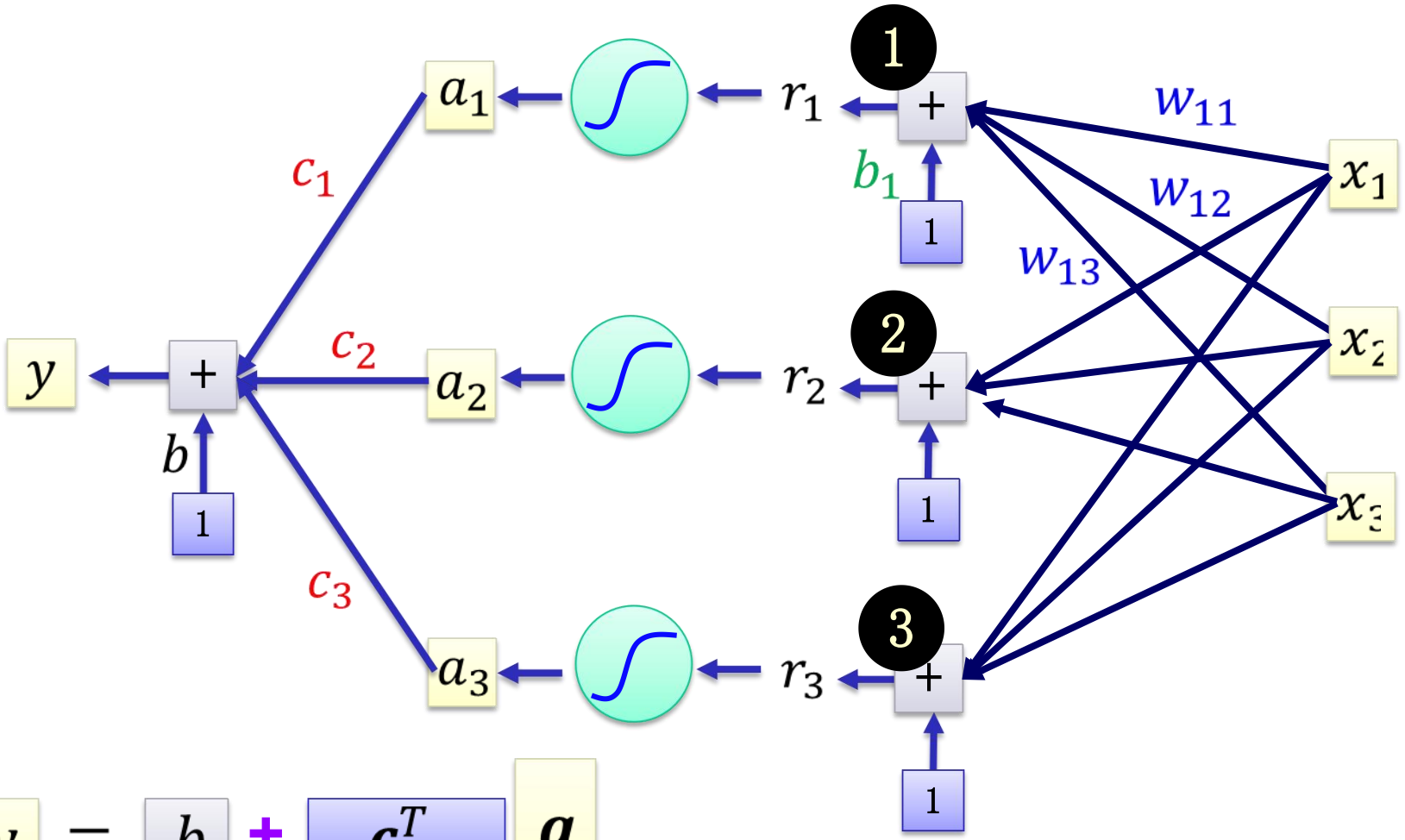
$$y = b + \sum_i c_i \, sigmoid\left(b_i + \sum_j w_{ij} x_j\right)$$

i: 1,2,3

j: 1,2,3

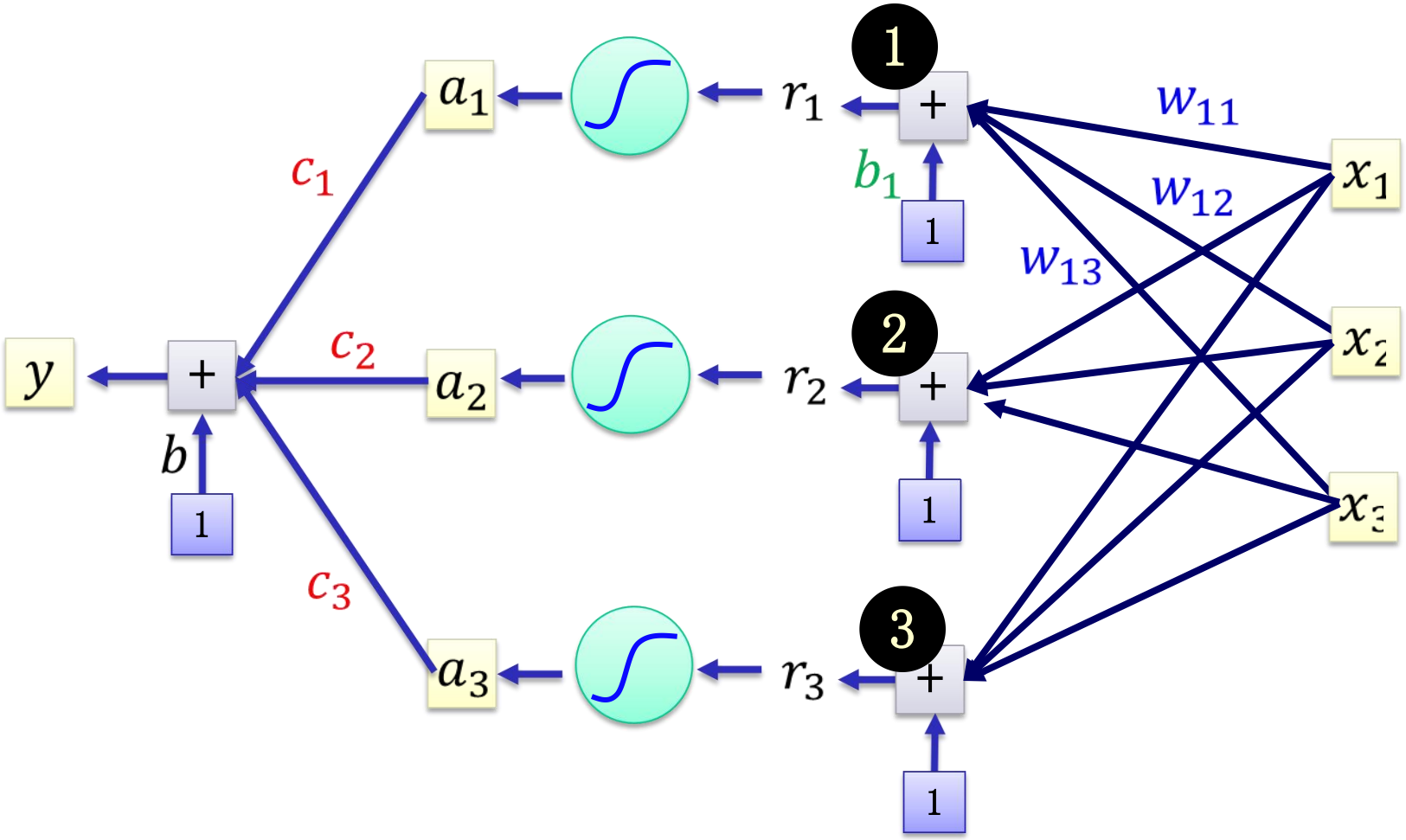


$$y = b + \boldsymbol{c}^T \boldsymbol{a}$$



$$y = b + \boldsymbol{c}^T \boldsymbol{a}$$

$$\boldsymbol{a} = \sigma(\boldsymbol{r}) \longleftrightarrow \boldsymbol{r} = \boldsymbol{b} + W\boldsymbol{x}$$



$$y = b + \boldsymbol{c}^T \sigma(\boldsymbol{b} + W\boldsymbol{x})$$

函数（含有未知参数）

$$y = b + c^T \sigma(b + Wx)$$

x feature

Unknown parameters

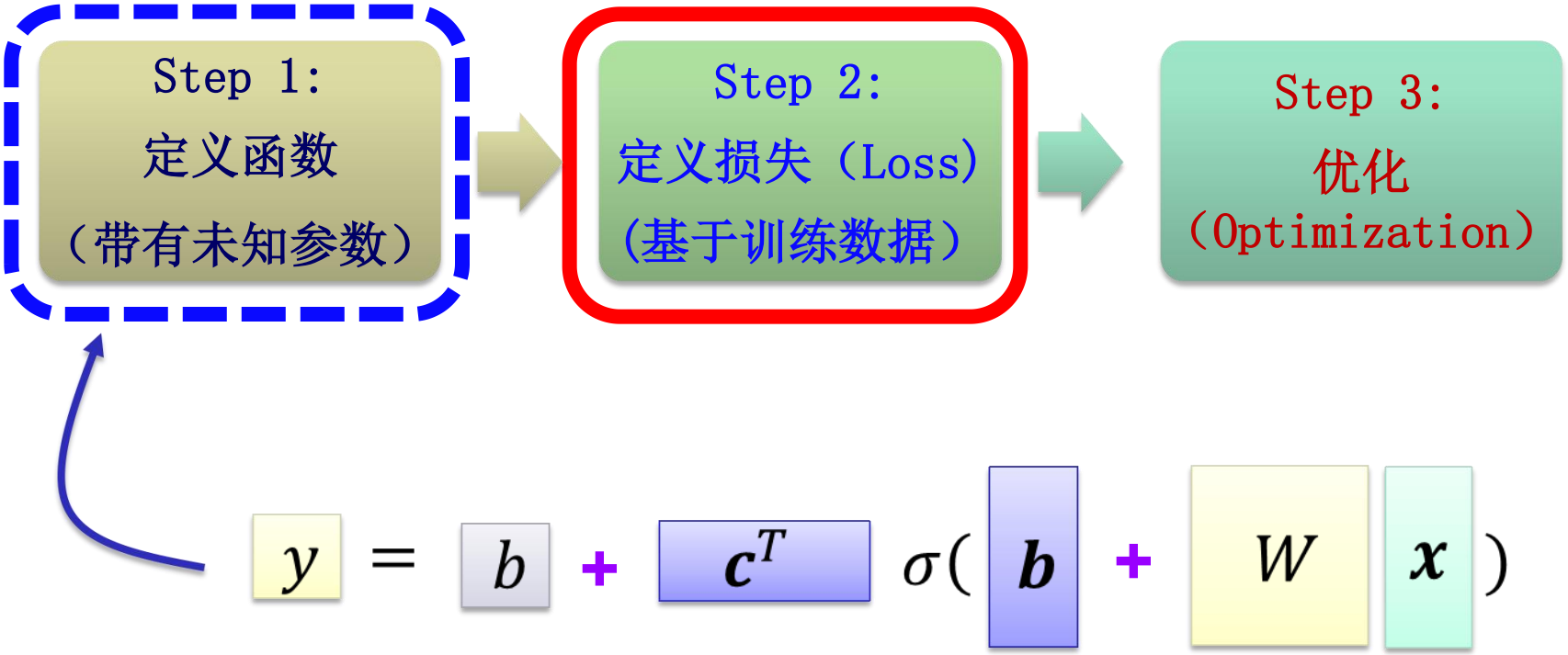
W b

c^T b

Rows of W

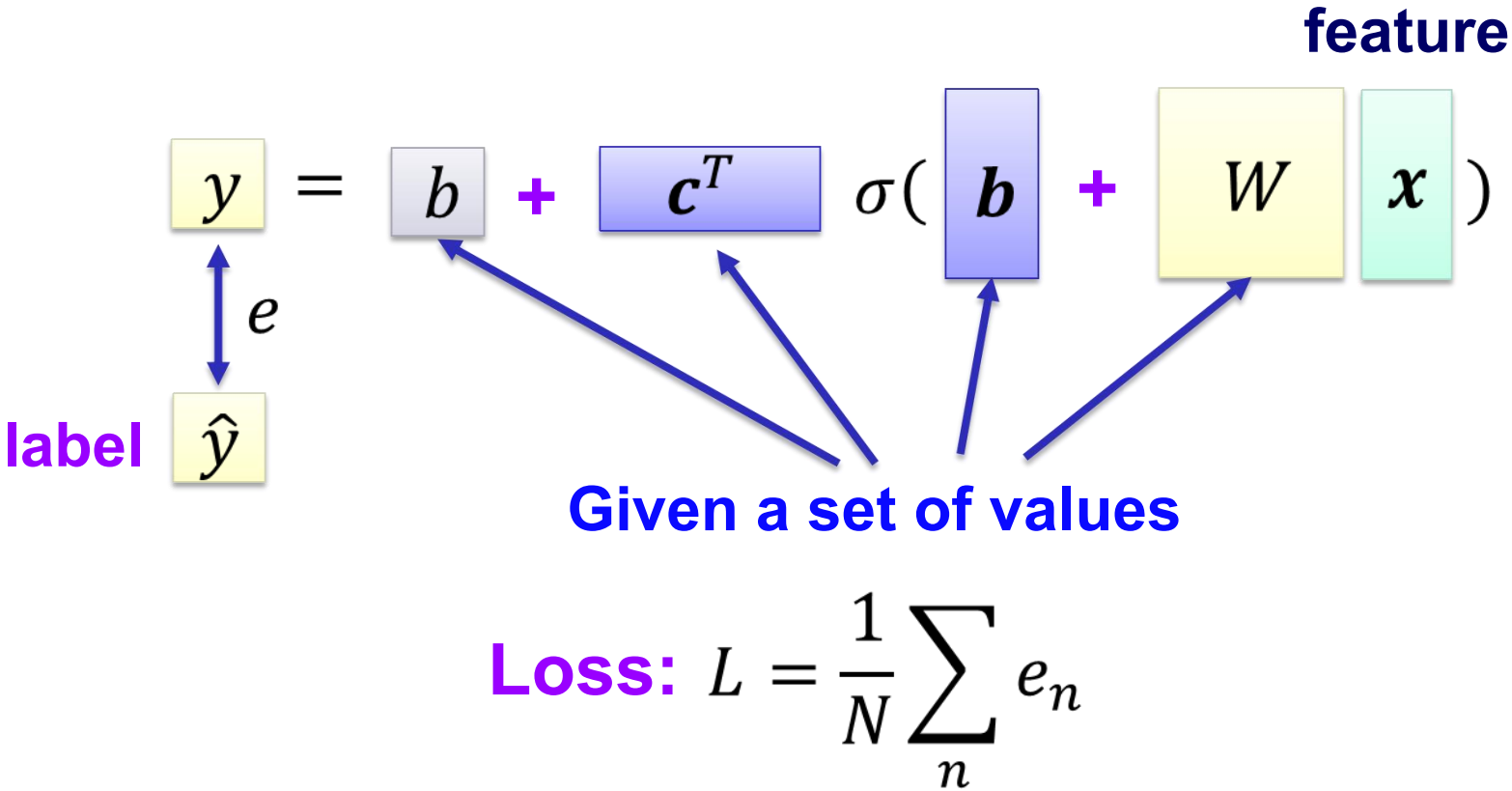
$$\theta = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \\ \vdots \end{bmatrix}$$

Back to ML Framework



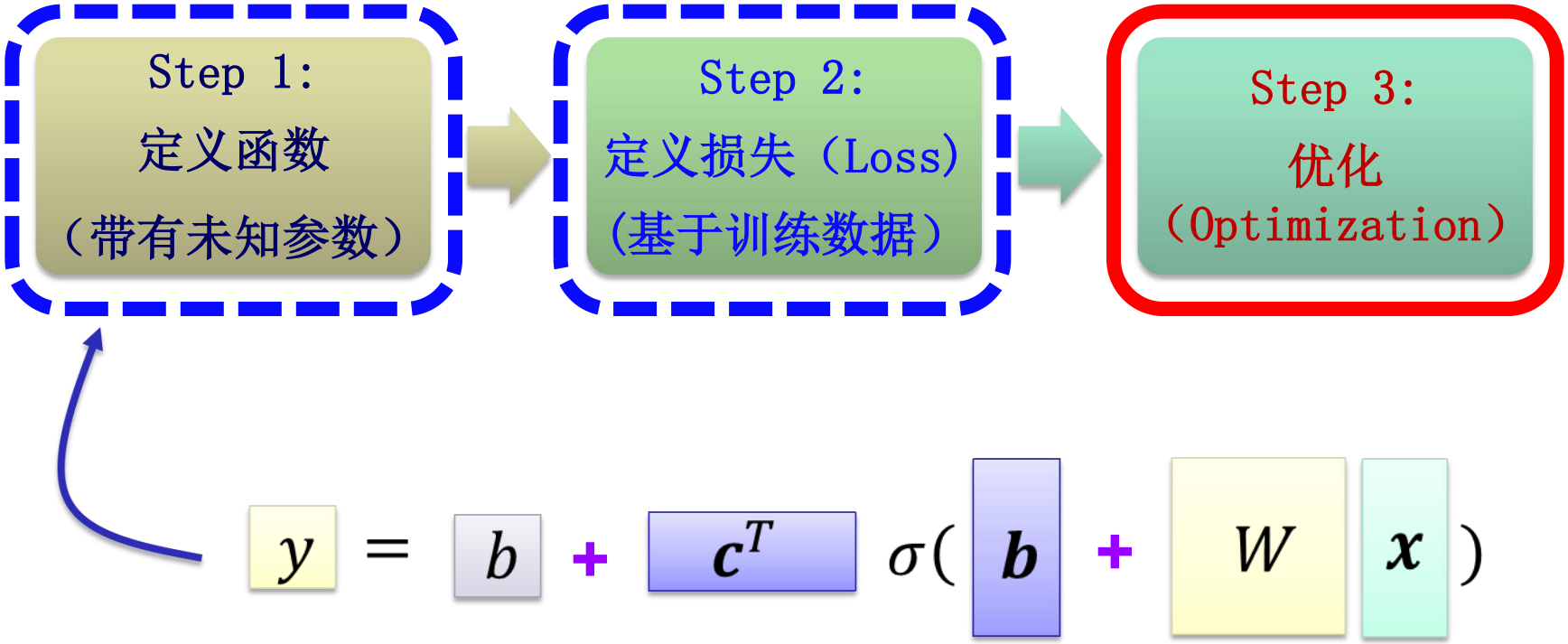
Loss（损失）

- 一个带有参数的函数： $L(\theta)$
- 评价一个模型参数组合的好坏



Loss: $L = \frac{1}{N} \sum_n e_n$

Back to ML Framework



模型优化

$$\boldsymbol{\theta}^* = arg\min_{\boldsymbol{\theta}} L$$

$$\boldsymbol{\theta} = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \\ \vdots \end{bmatrix}$$

- (Randomly) Pick initial values $\boldsymbol{\theta}^0$

$$\boldsymbol{g} = \begin{bmatrix} \frac{\partial L}{\partial \theta_1}|_{\boldsymbol{\theta}=\boldsymbol{\theta}^0} \\ \frac{\partial L}{\partial \theta_2}|_{\boldsymbol{\theta}=\boldsymbol{\theta}^0} \\ \vdots \end{bmatrix}$$

gradient

$$\begin{bmatrix} \theta_1^1 \\ \theta_2^1 \\ \vdots \end{bmatrix} \leftarrow \begin{bmatrix} \theta_1^0 \\ \theta_2^0 \\ \vdots \end{bmatrix} - \begin{bmatrix} \eta \frac{\partial L}{\partial \theta_1}|_{\boldsymbol{\theta}=\boldsymbol{\theta}^0} \\ \eta \frac{\partial L}{\partial \theta_2}|_{\boldsymbol{\theta}=\boldsymbol{\theta}^0} \\ \vdots \end{bmatrix}$$

$$\boldsymbol{g} = \nabla L(\boldsymbol{\theta}^0)$$

$$\boldsymbol{\theta}^1 \leftarrow \boldsymbol{\theta}^0 - \eta \boldsymbol{g}$$

模型优化

$$\boldsymbol{\theta}^* = arg\min_{\boldsymbol{\theta}} L$$

- (Randomly) Pick initial values $\boldsymbol{\theta}^0$
- Compute gradient $\boldsymbol{g} = \nabla L(\boldsymbol{\theta}^0)$

$$\boldsymbol{\theta}^1 \leftarrow \boldsymbol{\theta}^0 - \eta \boldsymbol{g}$$

- Compute gradient $\boldsymbol{g} = \nabla L(\boldsymbol{\theta}^1)$

$$\boldsymbol{\theta}^2 \leftarrow \boldsymbol{\theta}^1 - \eta \boldsymbol{g}$$

- Compute gradient $\boldsymbol{g} = \nabla L(\boldsymbol{\theta}^2)$

$$\boldsymbol{\theta}^3 \leftarrow \boldsymbol{\theta}^2 - \eta \boldsymbol{g}$$

模型优化

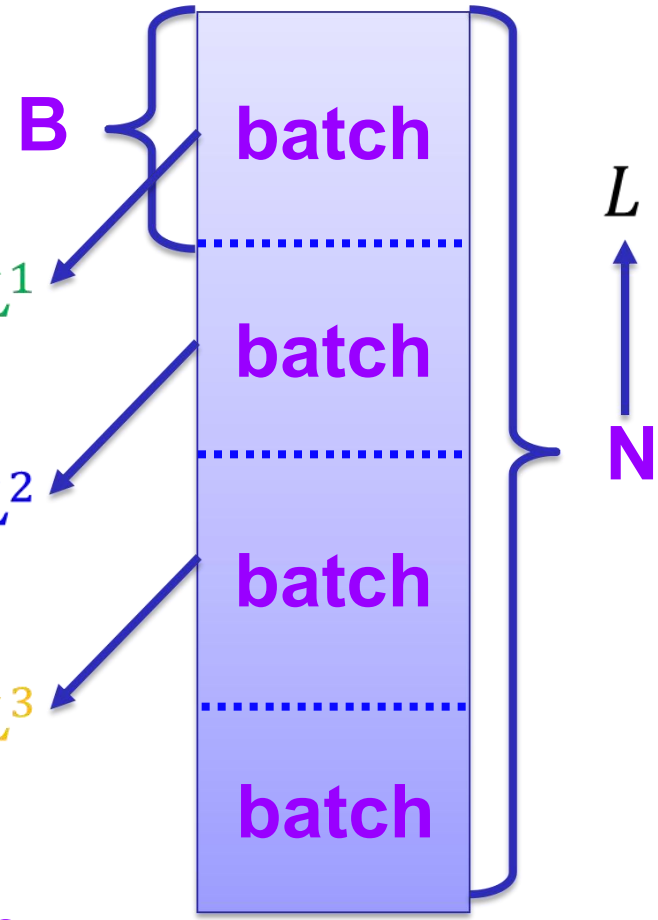
$$\boldsymbol{\theta}^* = arg\min_{\boldsymbol{\theta}} L$$

- (Randomly) Pick initial values $\boldsymbol{\theta}^0$
- Compute gradient $\boldsymbol{g} = \nabla L^1(\boldsymbol{\theta}^0)$ L^1

  update $\boldsymbol{\theta}^1 \leftarrow \boldsymbol{\theta}^0 - \eta \boldsymbol{g}$
- Compute gradient $\boldsymbol{g} = \nabla L^2(\boldsymbol{\theta}^1)$ L^2

  update $\boldsymbol{\theta}^2 \leftarrow \boldsymbol{\theta}^1 - \eta \boldsymbol{g}$
- Compute gradient $\boldsymbol{g} = \nabla L^3(\boldsymbol{\theta}^2)$ L^3

  update $\boldsymbol{\theta}^3 \leftarrow \boldsymbol{\theta}^2 - \eta \boldsymbol{g}$



1 epoch = see all the batches once

模型优化

Example 1

- 10,000 examples (N = 10,000)
- Batch size is 10 (B = 10)

How many update in 1 epoch?

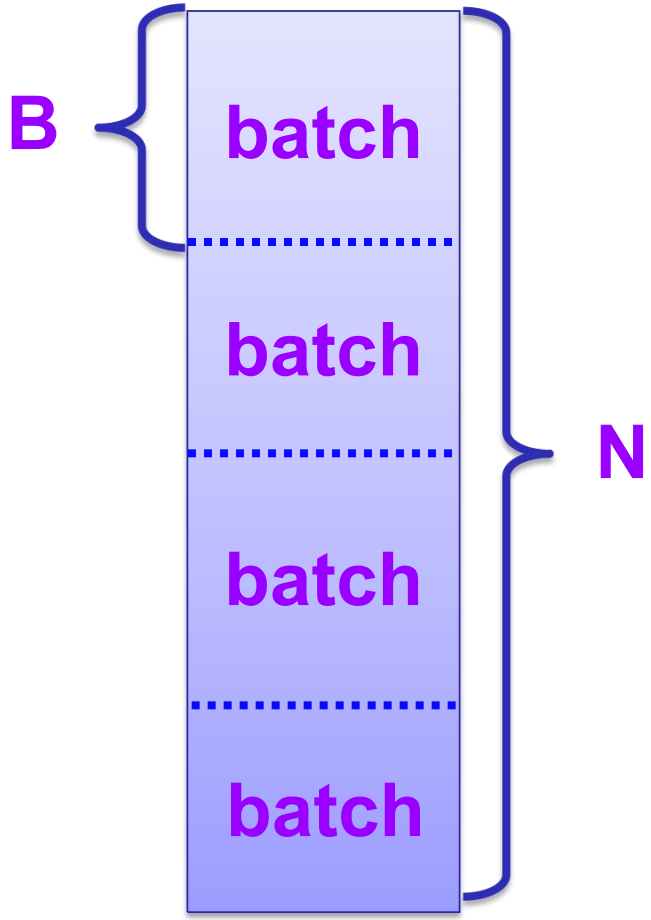
1,000 updates

Example 2

- 1,000 examples (N = 1,000)
- Batch size is 100 (B = 100)

How many update in 1 epoch?

10 updates



Back to ML Framework

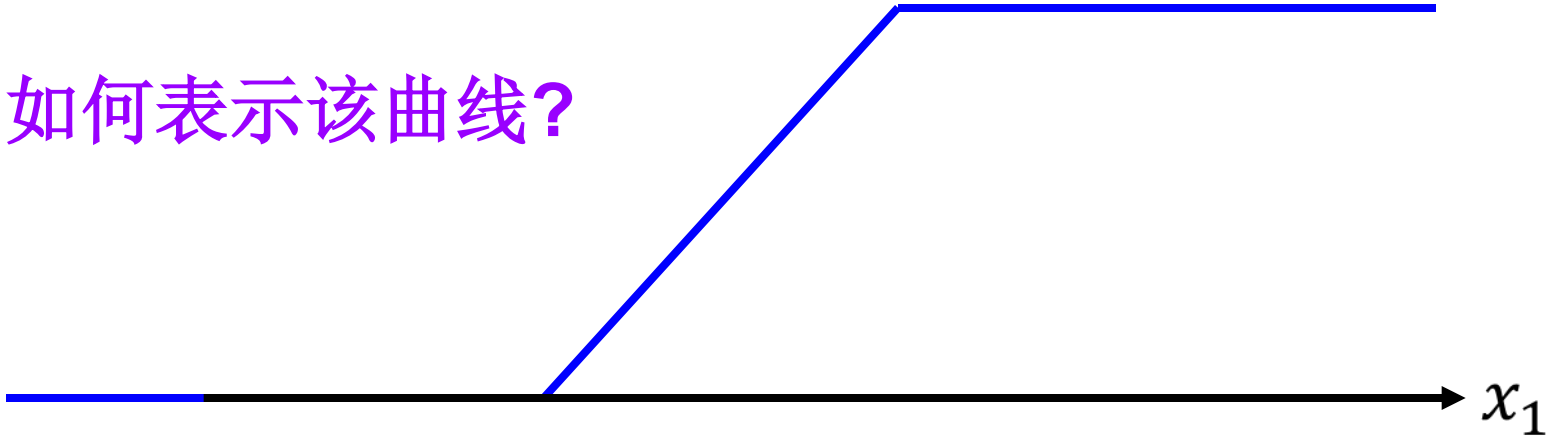


$$y = b + c^T \sigma(b + Wx)$$

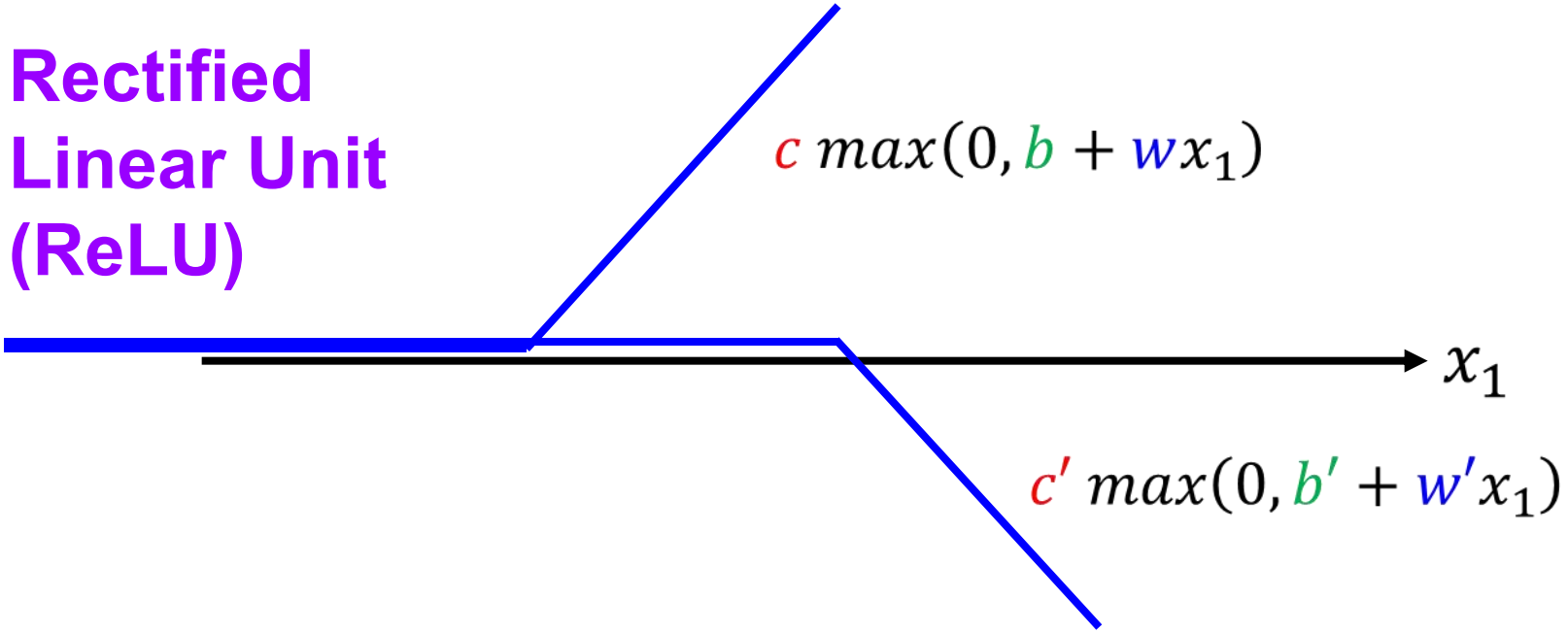
更多类型的模型 …

Sigmoid → ReLU

如何表示该曲线?



Rectified Linear Unit (ReLU)



Sigmoid → ReLU

$$y = b + \sum_{i} c_i \, sigmoid\left(b_i + \sum_{j} w_{ij} x_j\right)$$

Activation function

$$y = b + \sum_{2i} c_i \, max\left(0, b_i + \sum_{j} w_{ij} x_j\right)$$

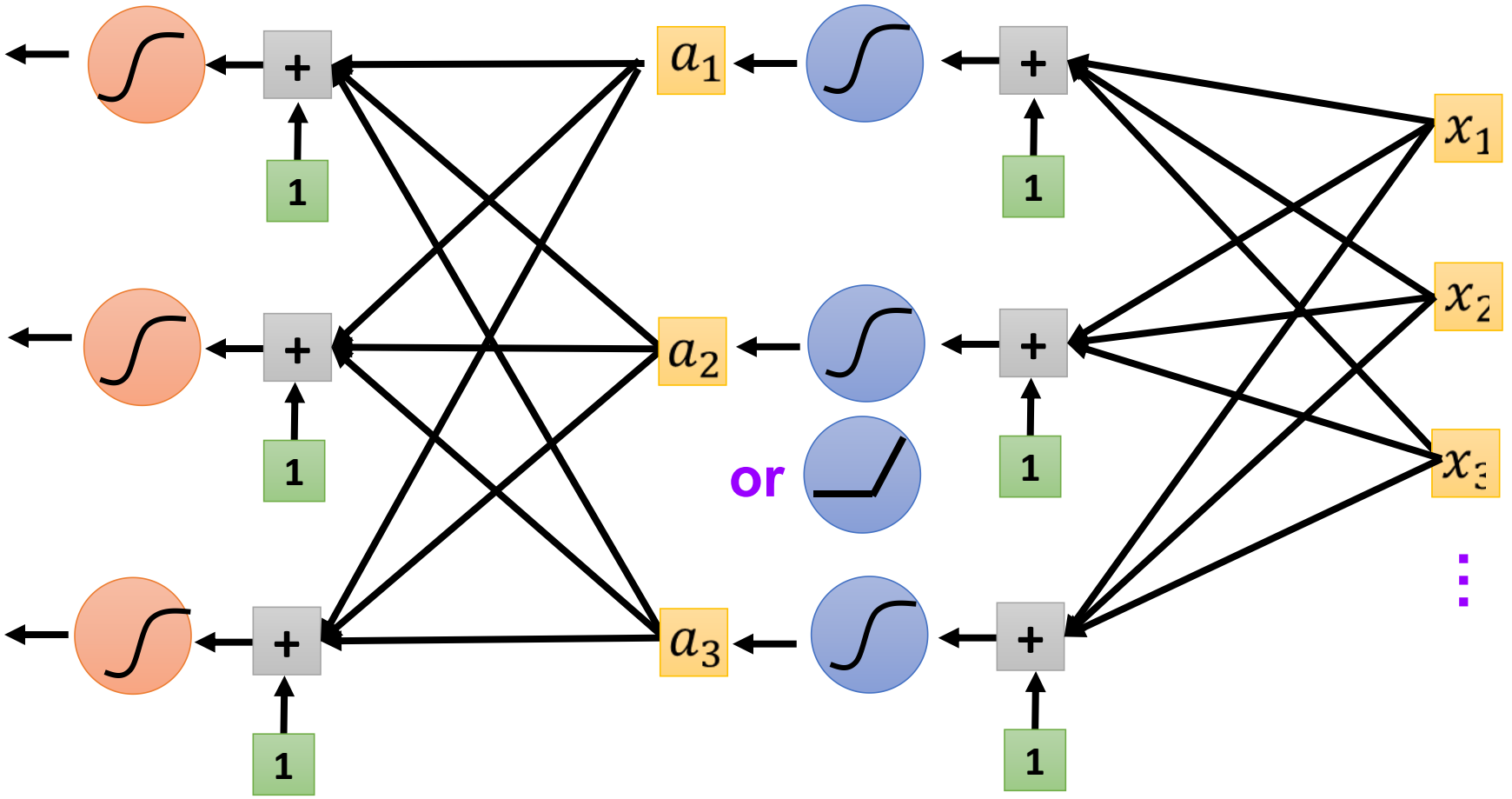
Which one is better?

Back to ML Framework



$$y = b + c^T \sigma(b + Wx)$$

更多类型的模型 …



$$a' = \sigma(b' + W' a) \qquad a = \sigma(b + W x)$$

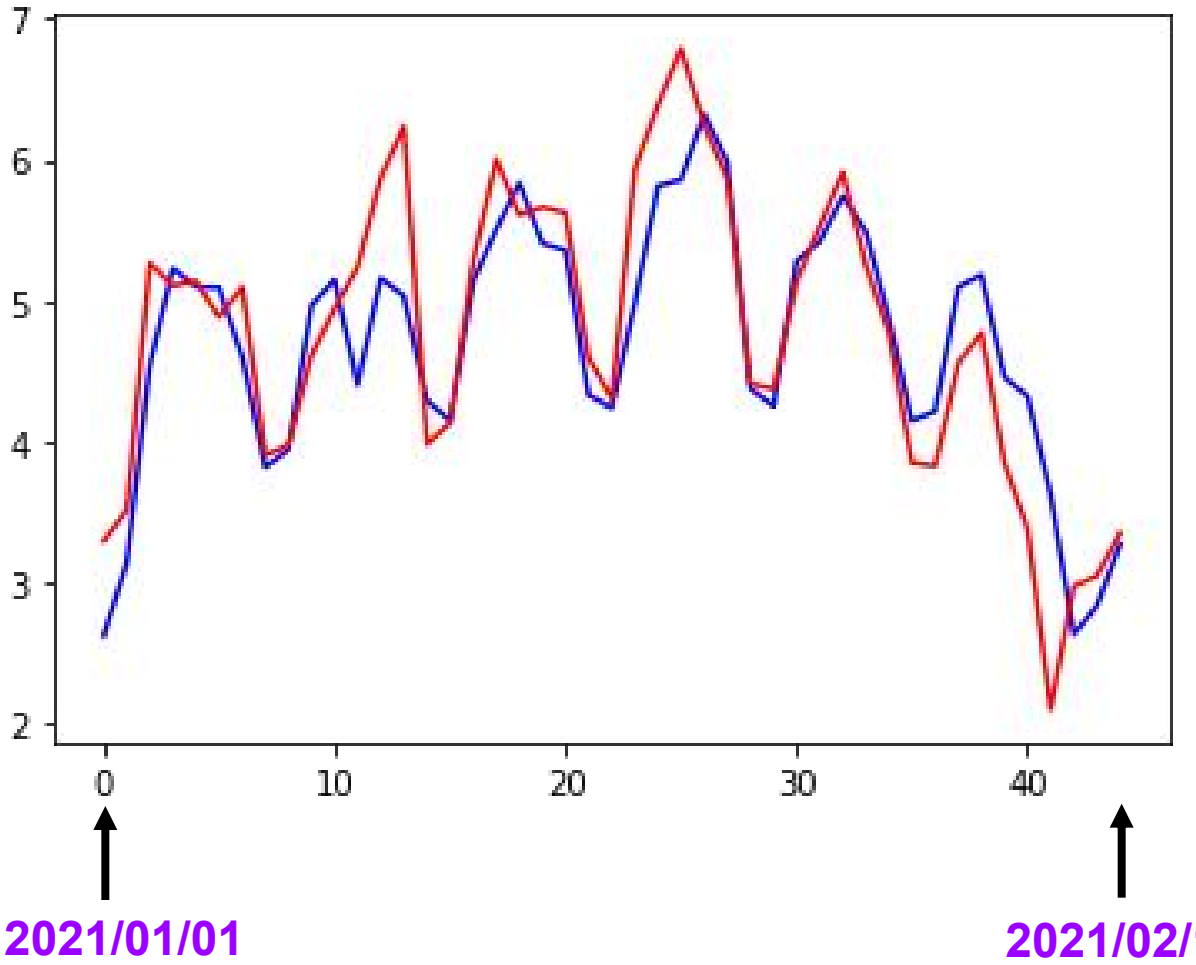
实验结果

- Loss for multiple hidden layers
  - 100 ReLU for each layer
  - 输入特征是过去 56 天的播放量

	1 layer
2017 – 2020	0.28k
2021	0.43k

3 layers

Red: 标签值
blue: 估计值

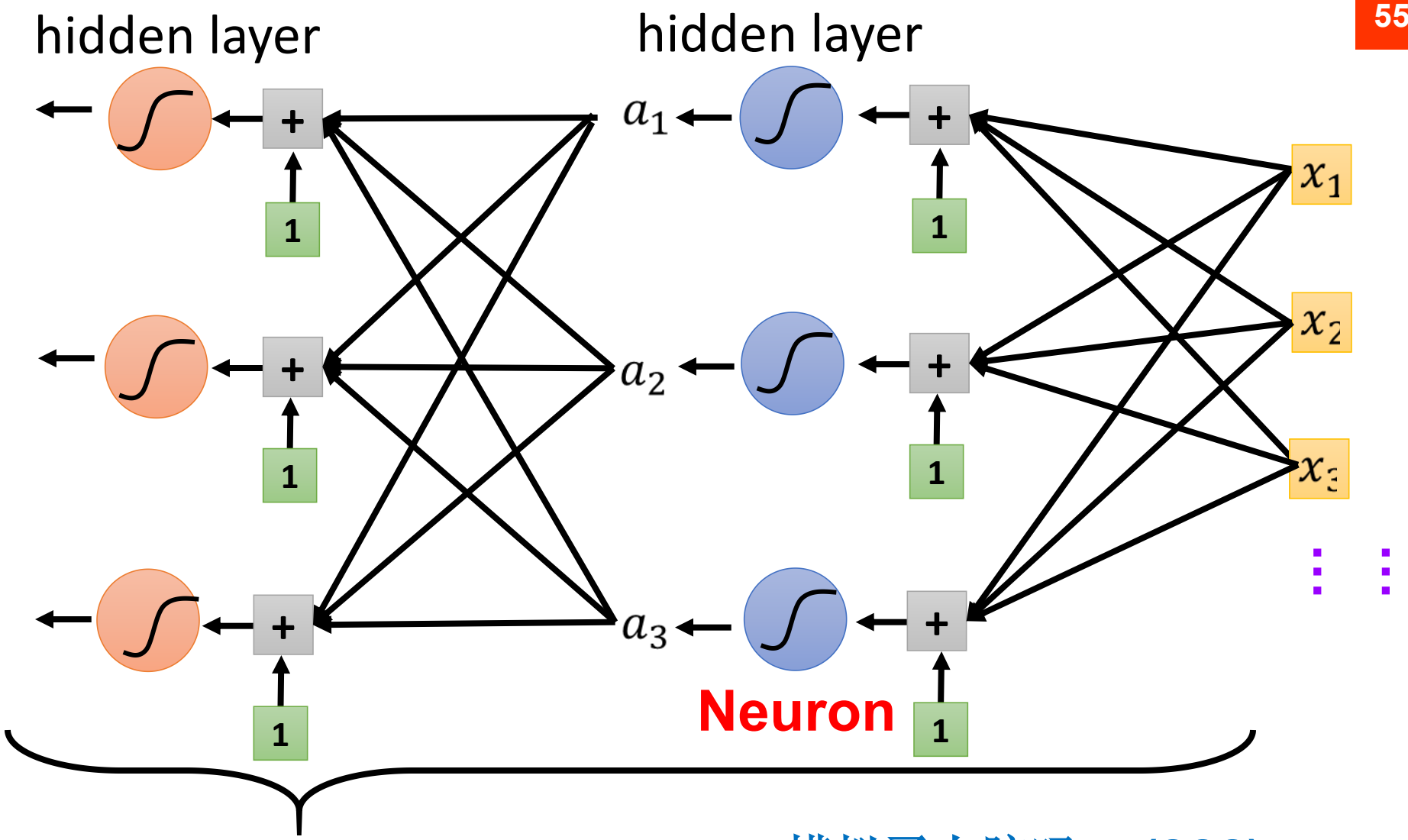


Back to ML Framework



$$y = b + c^T \sigma(b + Wx)$$

考虑起个美妙的名字吧！



模拟了人脑吗… (???)

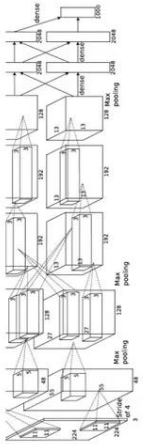
多个隐含层意味着 **Deep**

Deep Learning

Deep = Many hidden layers

8 layers

16.4%



AlexNet (2012)

19 layers

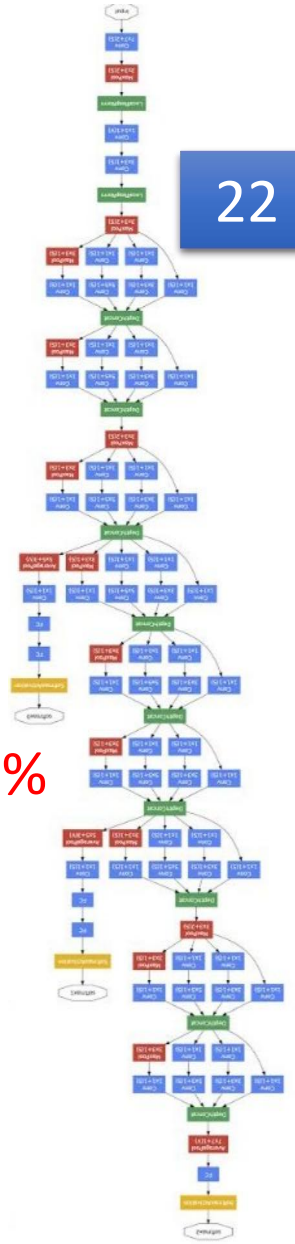
7.3%



VGG (2014)

22 layers

6.7%



GoogleNet (2014)

Deep = Many hidden layers

152 layers

119 layers

Special structure



Why we want "*Deep*" network, not "*Fat*" network?

16.4% AlexNet (2012)

7.3% VGG (2014)

6.7% GoogleNet (2014)

3.57% Residual Net (2015)

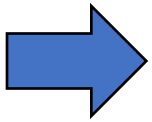
上海中心大厦 2015

为什么不考虑更深的网络呢?

- Loss for multiple hidden layers
  - 100 ReLU for each layer
  - 输入特征是过去56 的播放量

	1 layer	2 layer	3 layer
2017 – 2020	0.28k	0.18k	0.14k
2021	0.43k	0.39k	0.38k

Better on training data, worse on unseen data



Overfitting

模型选择

- 模型选择的准则?

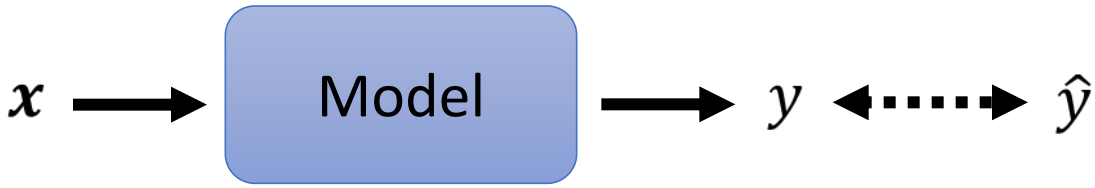
  以测试集的结果作为选择标准!

	1 layer	2 layer	3 layer	4 layer
2017 – 2020	0.28k	0.18k	0.14k	0.10k
2021	0.43k	0.39k	0.38k	0.44k

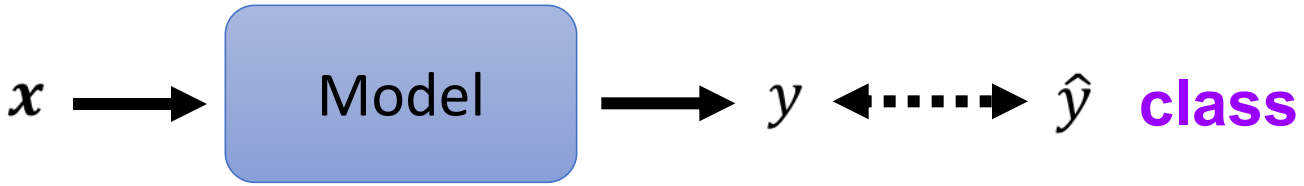
通用的多类分类器

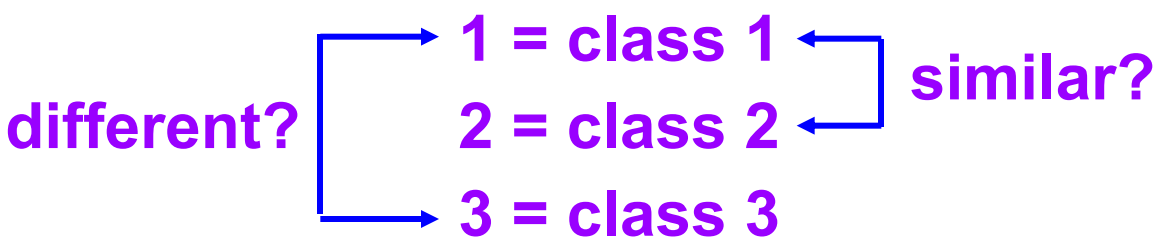
基于回归结果进行分类是否可行?

- Regression



- Classification as regression?

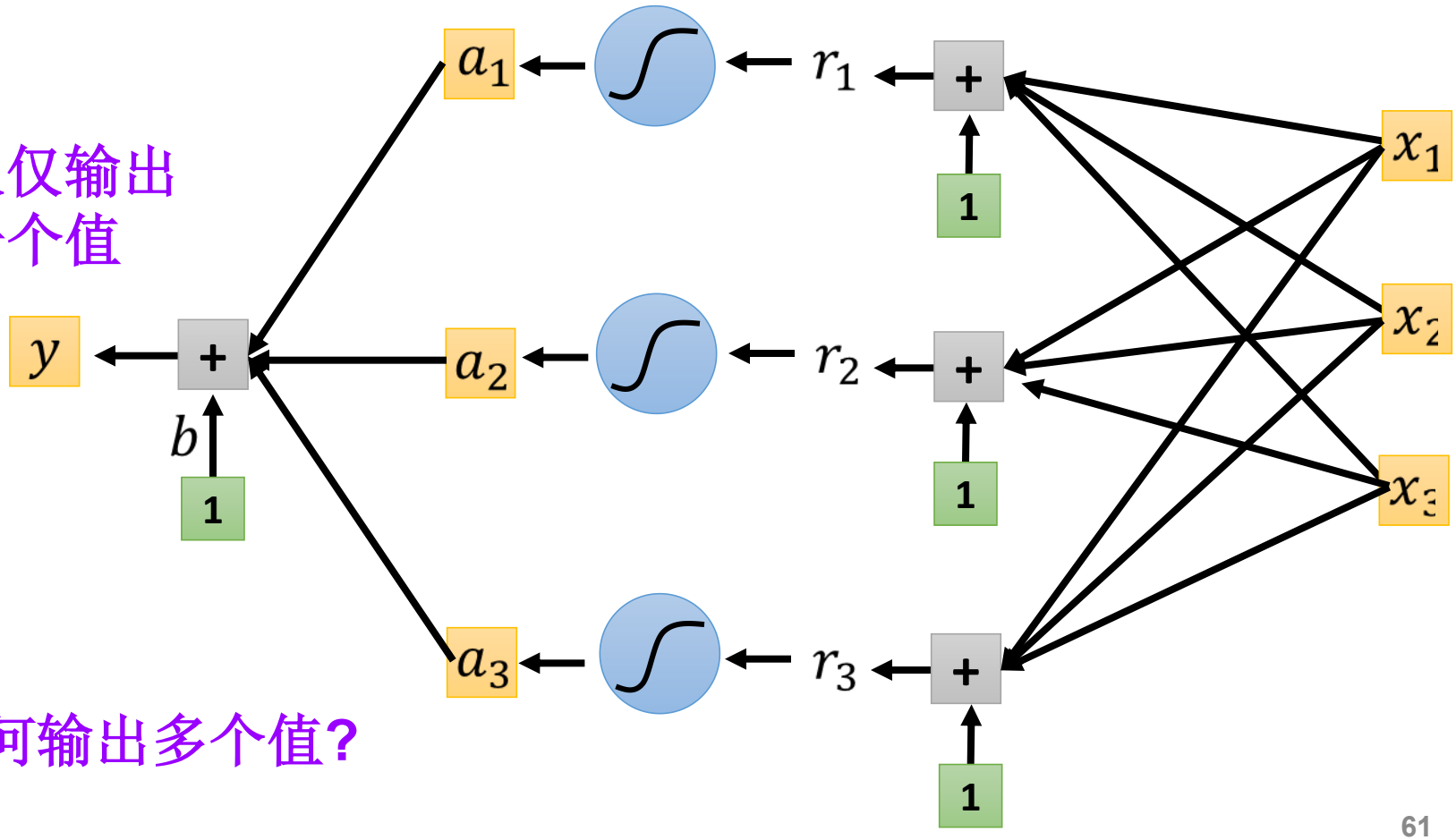




Class as one-hot vector

Class 1 Class 2 Class 3

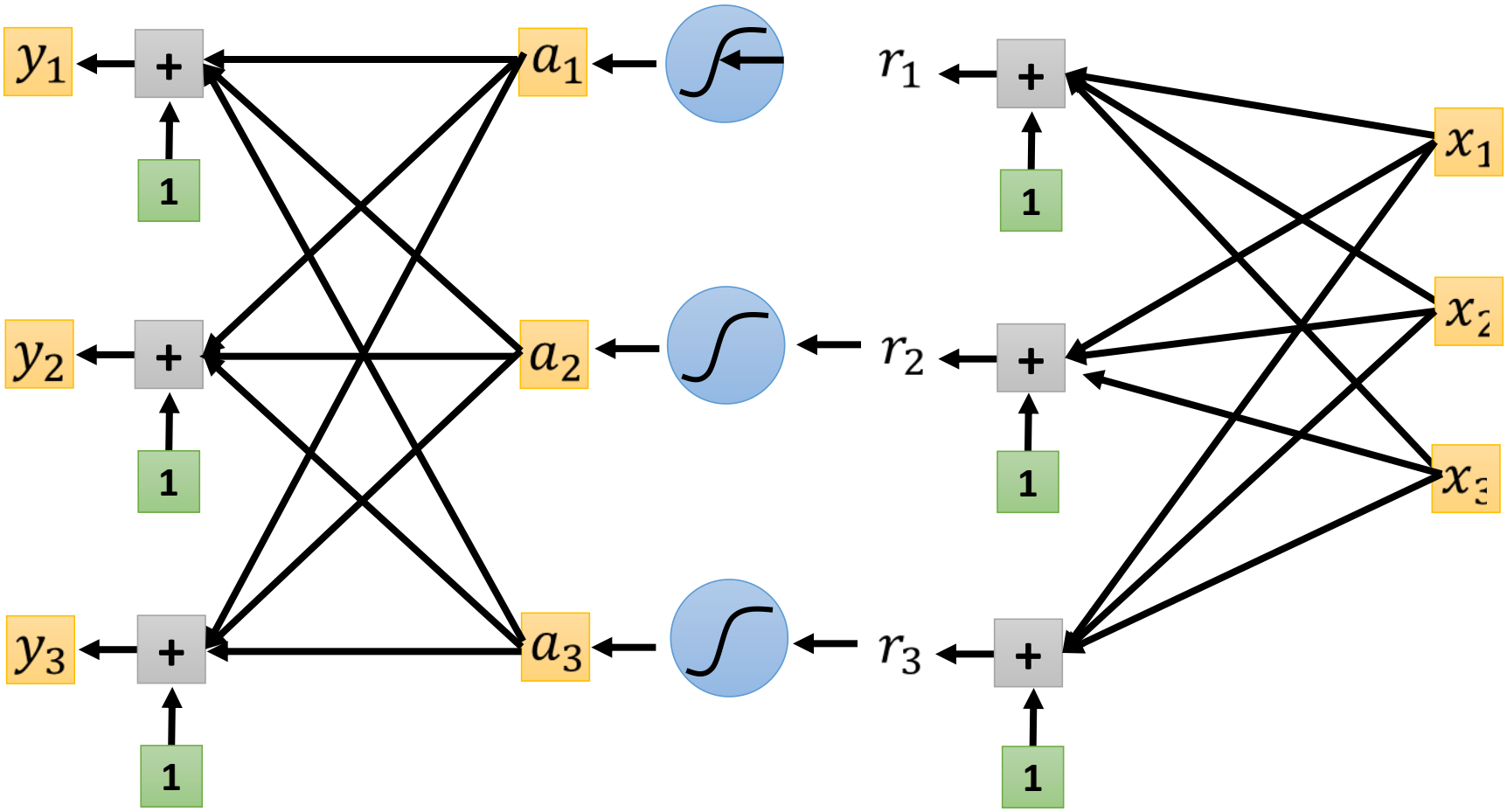
$$\hat{y} = \begin{bmatrix}1\\0\\0\end{bmatrix} \text{ or } \begin{bmatrix}0\\1\\0\end{bmatrix} \text{ or } \begin{bmatrix}0\\0\\1\end{bmatrix}$$



Class as one-hot vector

Class 1 Class 2 Class 3

$$\hat{y} = \begin{bmatrix}1\\0\\0\end{bmatrix} \text{ or } \begin{bmatrix}0\\1\\0\end{bmatrix} \text{ or } \begin{bmatrix}0\\0\\1\end{bmatrix}$$



Regression

feature

label

$$\hat{y} \longleftrightarrow y = b + c^T \sigma(b + W x)$$

Classification

feature

$$y = b' + W' \sigma(b + W x)$$

label

$$\hat{y} \longleftrightarrow y' = softmax(y)$$

0 or 1

使所有值
位于 [0, 1]

可能是
任意值

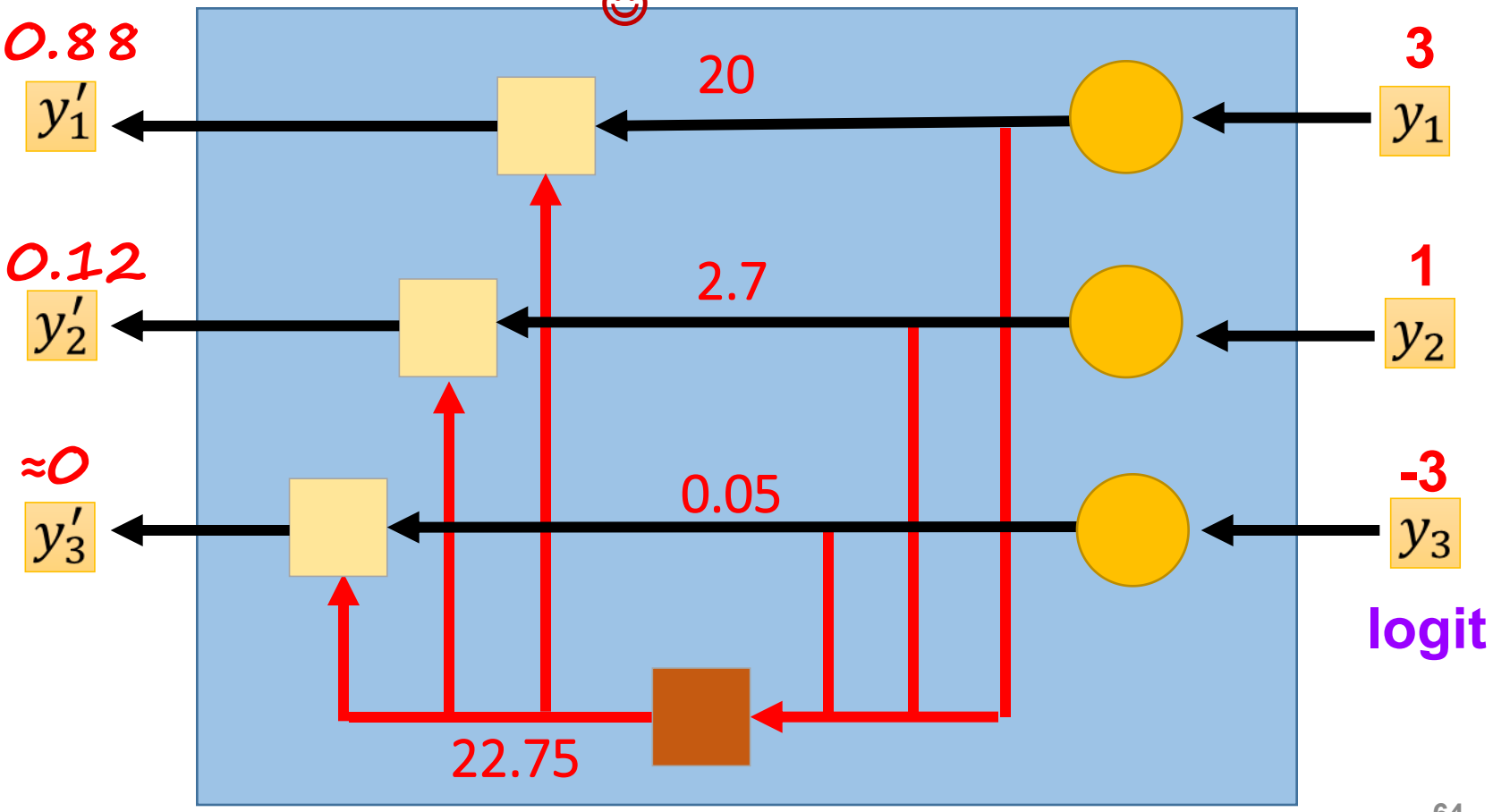
Soft-max

$$y_i' = \frac{exp(y_i)}{\sum_j exp(y_i)}$$

- $1 > y_i' > 0$
- $\sum_i y_i' = 1$

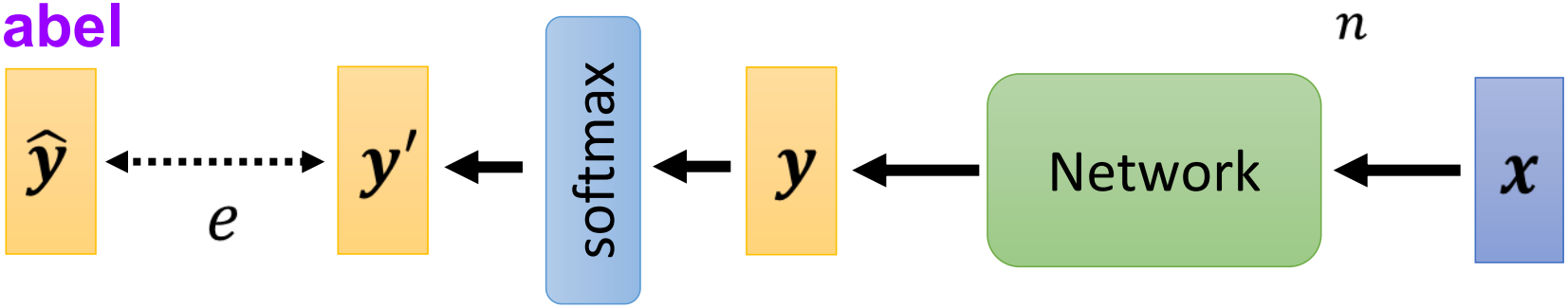
Softmax

How about binary classification? ☺



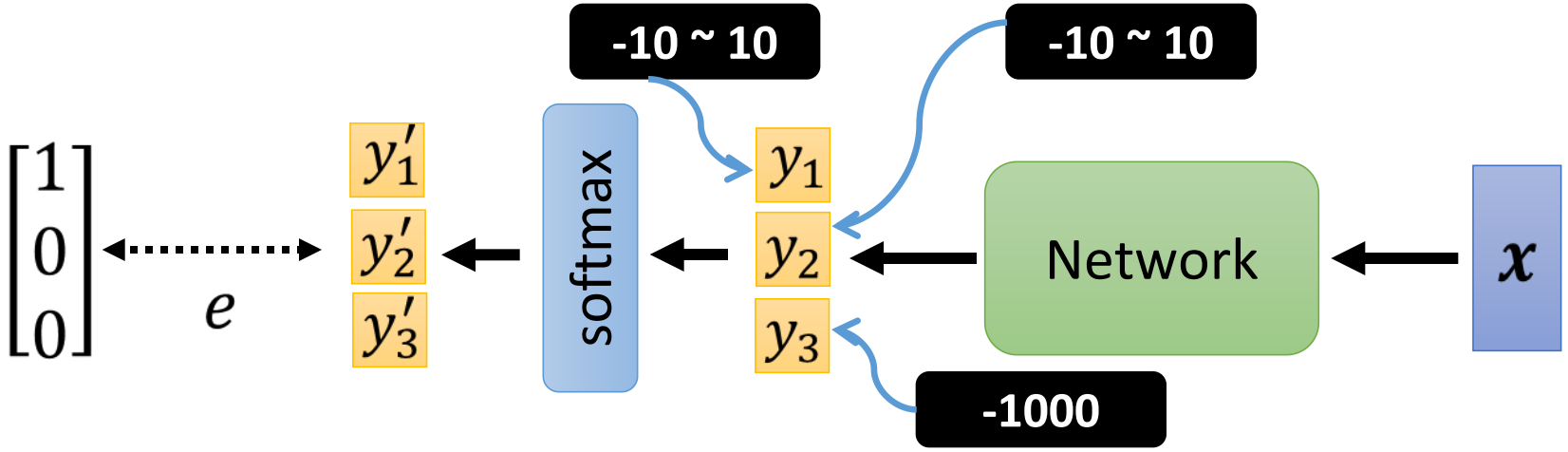
Loss of Classification

$$L = \frac{1}{N}\sum_{n} e_n$$

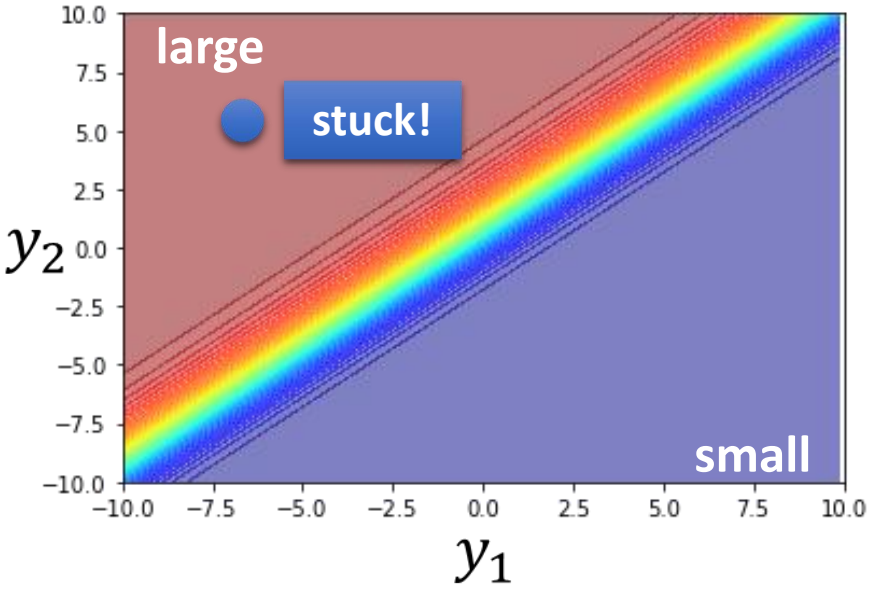


Mean Square Error (MSE) $e = \sum_i (\hat{y}_i - y'_i)^2$

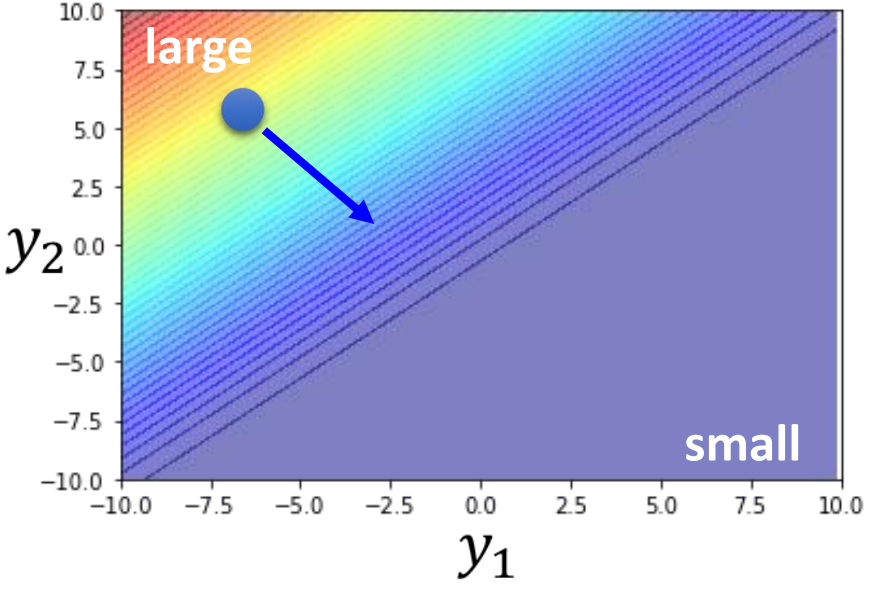
Cross-entropy WIN $e = -\sum_i \hat{y}_i ln y'_i$



Mean Square Error (MSE)



Cross-entropy



损失函数的形式可以影响模型优化的难度！